

فصل ۳. فرایندها

در این فصل، نگاهی دقیق‌تر خواهیم داشت به اینکه چگونه انواع مختلف فرایندها نقش اساسی در سیستم‌های توزیع‌شده ایفا می‌کنند. مفهوم فرایند از حوزه سیستم‌عامل‌ها نشأت می‌گیرد، جایی که معمولاً به‌عنوان یک برنامه در حال اجرا تعریف می‌شود. از دیدگاه سیستم‌عامل، مدیریت و زمان‌بندی فرایندها احتمالاً از مهم‌ترین مسائلی هستند که باید به آن‌ها رسیدگی شود. با این حال، وقتی صحبت از سیستم‌های توزیع‌شده می‌شود، مسائل دیگری نیز پدید می‌آیند که به همان اندازه یا حتی بیشتر اهمیت دارند.

بحث را با بررسی مفصل رشته‌ها (threads) و نقش آن‌ها در سیستم‌های توزیع‌شده آغاز می‌کنیم. همان‌طور که مشخص است، رشته‌ها نقش کلیدی در دستیابی به عملکرد بالا در محیط‌های چند هسته‌ای و چندپردازنده‌ای دارند، اما همچنین در ساختاردهی به کلاینت‌ها و سرورها نیز مؤثرند. در بسیاری موارد، مشاهده می‌شود که رشته‌ها جایگزین فرایندها شده‌اند و از خدمات سیستم‌عامل پایه برای تضمین حفاظت و تسهیل ارتباط استفاده می‌کنند. با این وجود، زمانی که پای عملکرد به میان می‌آید، رشته‌ها همچنان نقش مهمی ایفا می‌کنند.

در چند سال اخیر، مفهوم مجازی‌سازی (Virtualization) محبوبیت زیادی پیدا کرده است. مجازی‌سازی این امکان را به یک برنامه و حتی کل محیط عملیاتی آن (شامل سیستم‌عامل) می‌دهد که به‌طور هم‌زمان با برنامه‌های دیگر اجرا شود. این در حالی است که برنامه مذکور تا حد زیادی از سخت‌افزار و پلتفرم زیرساختی مستقل باقی می‌ماند و این ویژگی باعث افزایش قابل توجه قابلیت حمل‌پذیری می‌شود. افزون بر این، مجازی‌سازی در ایزوله‌سازی خرابی‌هایی که ناشی از خطاها یا مشکلات امنیتی هستند نیز کمک می‌کند. به همین دلیل، مجازی‌سازی مفهومی مهم در سیستم‌های توزیع‌شده محسوب می‌شود و به‌صورت جداگانه در این فصل به آن پرداخته خواهد شد.

سازمان‌دهی کلاینت-سرور از دیگر جنبه‌های مهم در سیستم‌های توزیع‌شده است. در این فصل، نگاهی دقیق خواهیم داشت به نحوه سازمان‌یابی کلاینت‌ها و سرورها. همچنین به اصول طراحی کلی سرورها توجه می‌کنیم، از جمله سرورهایی که به‌طور خاص در سیستم‌های توزیع‌شده شی‌گرا (object-based) استفاده می‌شوند. به‌طور ویژه به سرور وب معروف Apache می‌پردازیم. سازمان‌دهی خوشه‌های سرور نیز اهمیت دارد، به‌ویژه زمانی که این خوشه‌ها باید به‌صورت مشارکتی این تصور را ایجاد کنند که یک سیستم یکپارچه هستند. در این راستا، روش‌هایی مانند استفاده از سرورهای ناحیه گسترده نظیر PlanetLab مورد بحث قرار می‌گیرند.

یکی از مسائل مهم دیگر، به‌ویژه در سیستم‌های توزیع‌شده گسترده، انتقال فرایندها میان ماشین‌های مختلف است. مهاجرت کد (code migration) به‌طور خاص می‌تواند در مقیاس‌پذیری مؤثر باشد، و نیز به پیکربندی پویا و توزیع کلاینت‌ها و سرورها کمک کند. اینکه واقعاً منظور از مهاجرت کد چیست و چه تأثیراتی دارد، نیز در این فصل بررسی می‌شود.

۳.۱ رشته‌ها (Threads)

اگرچه فرایندها به‌عنوان یکی از اجزای اساسی در ساخت سیستم‌های توزیع‌شده محسوب می‌شوند، تجربه عملی نشان داده است که میزان دقت یا گرانولاریتی‌ای (سطح تفکیک) که فرایندها توسط سیستم‌عامل‌ها فراهم می‌کنند، برای ساخت سیستم‌های توزیع‌شده کافی نیست. در عوض، مشخص شده است که داشتن سطح تفکیک دقیق‌تر، به‌صورت استفاده از چند نخ کنترل در هر فرایند، ساخت برنامه‌های توزیع‌شده را بسیار ساده‌تر می‌کند. و برای دستیابی به عملکرد بهتر، در این بخش نگاهی دقیق‌تر می‌اندازیم به نقش رشته‌ها در سیستم‌های توزیع‌شده و بررسی می‌کنیم که چرا آن‌ها این قدر اهمیت دارند. اطلاعات بیشتر درباره رشته‌ها و نحوه استفاده از آن‌ها برای ساخت برنامه‌ها را می‌توان در منابع زیر یافت (Herlihy and Shavit, ۲۰۰۸; Lewis and Berg, ۱۹۹۸; Stevens, ۱۹۹۹; Robbins and Robbins, ۲۰۰۳). مطالعه این منبع به‌ویژه برای آشنایی با برنامه‌نویسی همروند چندنخی توصیه می‌شود.

۳.۱.۱ معرفی رشته‌ها

برای درک نقش رشته‌ها در سیستم‌های توزیع‌شده، ابتدا باید بفهمیم که «فرایند» چیست و رابطه بین فرایندها و رشته‌ها چگونه است. برای اجرای یک برنامه، سیستم‌عامل مجموعه‌ای از فرایندهای مجازی ایجاد می‌کند که هر یک برای اجرای یک برنامه متفاوت به‌کار می‌رود. برای پیگیری این فرایندهای

مجازی، سیستم‌عامل یک جدول فرایند (Process Table) نگهداری می‌کند که شامل اطلاعاتی مانند مقادیر ثبات‌های CPU، نگاشت حافظه، فایل‌های باز، اطلاعات حسابداری، سطوح دسترسی و ... است. این اطلاعات به‌صورت مشترک تشکیل‌دهنده «زمینه فرایند» یا همان Process Context هستند. زمینه فرایند را می‌توان هم‌ارز نرم‌افزاری زمینه پردازنده سخت‌افزاری دانست. این زمینه شامل حداقل اطلاعاتی است که سخت‌افزار برای رسیدگی به وقفه‌ها ذخیره کرده و سپس برای ادامه اجرای فرایند، آن‌ها را بازیابی می‌کند. این اطلاعات حداقل شامل شمارنده برنامه (Program Counter) بوده و ممکن است شامل مقادیر سایر ثبات‌ها مانند اشاره‌گر پشته (Stack Pointer) نیز باشد.

فرایند معمولاً به‌عنوان یک برنامه در حال اجرا تعریف می‌شود؛ یعنی برنامه‌ای که در حال حاضر در یکی از پردازنده‌های مجازی سیستم‌عامل اجرا می‌شود. یکی از مسائل مهم در اینجا آن است که سیستم‌عامل تلاش زیادی می‌کند تا اطمینان حاصل کند که فرایندهای مستقل، چه از روی قصد و چه سهواً، رفتار یکدیگر را مختل نکنند. به‌عبارت دیگر، این‌که چندین فرایند به‌طور هم‌زمان از یک CPU یا سایر منابع سخت‌افزاری استفاده کنند، باید برای هر فرایند نامرئی باشد. معمولاً برای اعمال این جداسازی، نیاز به پشتیبانی سخت‌افزاری وجود دارد.

البته این شفافیت هم‌زمانی، بهایی نیز دارد. برای مثال، هر بار که یک فرایند ایجاد می‌شود، سیستم‌عامل باید یک فضای آدرس مستقل و کامل برای آن تخصیص دهد. این تخصیص می‌تواند شامل مقداردهی اولیه به بخش‌هایی از حافظه، مانند بخش داده، کپی برنامه به یک بخش آزمایشی، و ایجاد پشته‌ای برای داده‌های موقتی باشد. به‌طور مشابه، جابه‌جایی CPU بین دو فرایند نیز ممکن است به منابعی نیاز داشته باشد. علاوه بر ذخیره‌سازی داده‌های وضعیت (مانند شمارنده برنامه و اشاره‌گر پشته)، سیستم‌عامل باید ثبات‌های ماژول مدیریت حافظه (MMU) را نیز بازنویسی کرده و کش‌های ترجمه آدرس، مانند TLB را نامعتبر سازد. همچنین اگر سیستم‌عامل بیش از ظرفیت حافظه اصلی، از فرایندها پشتیبانی کند، ممکن است نیاز به جابه‌جایی فرایندها بین حافظه اصلی و دیسک داشته باشد، قبل از اینکه بتواند فرایند جدیدی را فعال کند.

درست مانند یک فرایند، یک نخ نیز بخش خاصی از کد را به‌طور مستقل از سایر رشته‌ها اجرا می‌کند. با این حال، برخلاف فرایندها، در مورد رشته‌ها تلاشی برای دستیابی به سطح بالایی از شفافیت هم‌زمانی (concurrency transparency) صورت نمی‌گیرد، زیرا این کار می‌تواند باعث افت عملکرد شود. بنابراین، یک سیستم نخ (thread system) به‌طور کلی تنها اطلاعات حداقلی لازم برای اجازه اشتراک‌گذاری CPU بین چندین نخ را نگه می‌دارد. به‌طور خاص، زمینه نخ (thread context) اغلب شامل چیزی جز زمینه پردازنده (processor context) به‌همراه برخی اطلاعات مربوط به مدیریت نخ نیست. به‌عنوان مثال، یک سیستم نخ ممکن است وضعیت اسناد یک نخ بر روی یک متغیر mutex را نگه دارد تا نخ مسدودشده برای اجرا انتخاب نشود. اطلاعاتی که برای مدیریت چندین نخ به‌طور دقیق ضروری نیستند، معمولاً نادیده گرفته می‌شوند. به همین دلیل، محافظت از داده‌ها در برابر دسترسی‌های نامناسب رشته‌ها در درون یک فرایند، به‌طور کامل بر عهده توسعه‌دهندگان برنامه قرار دارد. بنابراین، می‌توان گفت که زمینه پردازنده در زمینه نخ قرار دارد و این زمینه نخ نیز درون زمینه فرایند قرار می‌گیرد.

دو پیامد مهم برای استفاده از رشته‌ها وجود دارد، همان‌طور که پیش‌تر بیان شد. یک، عملکرد یک برنامه چندنخی (multithreaded) در اغلب مواقع لزوماً بدتر از نسخه تک‌نخی آن نیست. در واقع، استفاده از چند نخ می‌تواند منجر به بهبود عملکرد نیز شود. دوم، از آنجا که رشته‌ها به‌صورت خودکار در برابر یکدیگر محافظت نمی‌شوند، توسعه برنامه‌های چندنخی به تلاش ذهنی بیشتری نیاز دارد. طراحی درست و ساده‌نگه‌داشتن امور، بسیار مهم است؛ هرچند در عمل، اغلب این اصل رعایت نمی‌شود.

استفاده از رشته‌ها در سیستم‌های غیر توزیع‌شده

پیش از بررسی نقش رشته‌ها در سیستم‌های توزیع‌شده، ابتدا باید نگاهی به کاربرد رشته‌ها در سیستم‌های سنتی و غیرتوزیع‌شده داشته باشیم. چندین مزیت برای استفاده از فرایندهای چندنخی در این نوع سیستم‌ها وجود دارد که محبوبیت آن‌ها را افزایش داده‌اند.

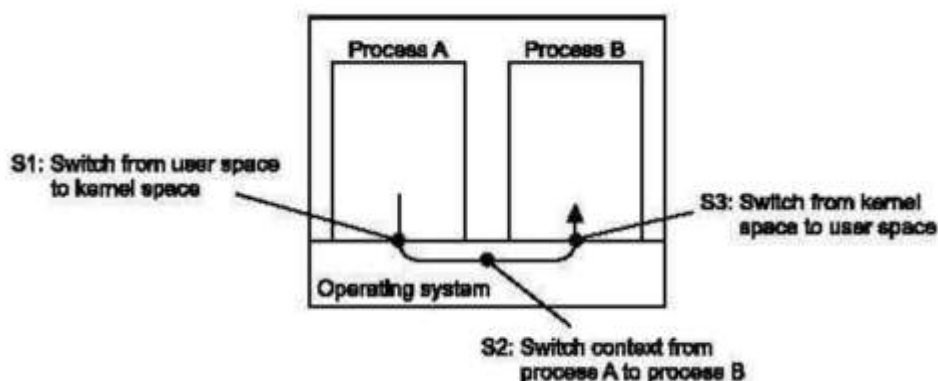
مهم‌ترین مزیت از این حقیقت میاد که در فرایندهای تک‌نخی، زمانی که یک فراخوان سیستمی مسدودکننده (blocking system call) اجرا می‌شود، کل فرایند مسدود می‌شود. برای مثال، فرض کنید یک برنامه مانند نرم‌افزار صفحه‌گسترده (spreadsheet) قرار است اجازه دهد کاربر به‌صورت تعاملی و هم‌زمان داده‌ها را تغییر دهد. یکی از ویژگی‌های کلیدی این نرم‌افزارها، حفظ وابستگی‌های فرمول‌ها بین سلول‌های مختلف است. اگر یک سلول تغییر کند، باید سایر سلول‌های وابسته نیز به‌روزرسانی شوند. در حالت تک‌نخی، این کار باعث توقف کل فرایند در هنگام ورود داده توسط کاربر می‌شود.

اما با استفاده از چند نخ، می‌توان اجرای این عملیات‌ها را جدا کرد. به عنوان مثال، یک نخ برای دریافت ورودی از کاربر، یک نخ برای به‌روزرسانی محاسبات، و نخ دیگری برای ذخیره‌سازی اطلاعات در دیسک در نظر گرفت. این مدل امکان پردازش موازی و مؤثر را فراهم می‌کند.

یکی دیگر از مزایای برنامه‌های چندنخی (multithreading) این است که امکان بهره‌برداری از موازی‌سازی (parallelism) هنگام اجرای برنامه بر روی سیستم‌های چندپردازنده‌ای (multiprocessor) یا چند هسته‌ای (multicore) را فراهم می‌سازد. در چنین حالتی، هر نخ به یک CPU یا هسته متفاوت اختصاص داده می‌شود، در حالی که داده‌های اشتراکی در حافظه مشترک نگهداری می‌شوند. زمانی که این طراحی به‌درستی انجام شود، این نوع موازی‌سازی می‌تواند شفاف باشد؛ یعنی فرایند می‌تواند حتی روی یک سیستم تک‌پردازنده نیز اجرا شود، هر چند با سرعت کمتر.

چندنخی برای موازی‌سازی با دسترسی گسترده‌تر به سیستم‌های چند هسته‌ای و چندپردازنده‌ای ارزان‌قیمت، اهمیت بیشتری یافته است. این نوع سیستم‌ها معمولاً برای اجرای سرورها در برنامه‌های سمت کاربر استفاده می‌شوند، ولی امروزه به‌طور گسترده‌ای در دستگاه‌هایی مانند تلفن‌های هوشمند نیز کاربرد دارند. چندنخی همچنین در زمینه برنامه‌های بزرگ نیز بسیار مفید است. این برنامه‌ها اغلب به‌صورت مجموعه‌ای از برنامه‌های همکار (cooperating programs) طراحی می‌شوند که هر کدام توسط یک فرایند جداگانه اجرا می‌گردند. این رویکرد در محیط‌های یونیکس رایج است. همکاری بین برنامه‌ها معمولاً از طریق مکانیزم‌های ارتباط بین فرایندی (IPC: Interprocess Communication) انجام می‌شود. در سیستم‌های یونیکس، این مکانیزم‌ها معمولاً شامل پایپ‌های نام‌دار (named pipes)، صف پیام (message queues)، و بخش‌های حافظه اشتراکی (shared memory segments) هستند. (به منابع Stevens & Rago {۲۰۰۵} مراجعه کنید).

نکته منفی اصلی این است که همه مکانیزم‌های IPC معمولاً نیاز به تعویض زمینه (context switching) دارند، که در شکل زیر در سه نقطه مختلف نمایش داده شده است:



شکل ۳.۱: تعویض زمینه به عنوان نتیجه‌ای از IPC

- سوئیچ از فضای کاربر به فضای کرنل: S۱
- B به فرایند A تعویض زمینه از فرایند: S۲
- سوئیچ از فضای کرنل به فضای کاربر: S۳

از آنجایی که IPC نیاز به مداخله (کرنل) دارد، معمولاً یک فرایند ابتدا باید از حالت کاربر به حالت کرنل منتقل شود، همان‌طور که در S۱ در شکل ۳.۱ نشان داده شده است. این کار نیاز به تغییر نگاشت حافظه در MMU دارد، همچنین باید TLB نیز پاک‌سازی شود. در داخل کرنل، تعویض زمینه بین فرایندها انجام می‌شود (در S۲ در شکل)، و پس از آن فرایند دیگر باید دوباره از حالت کرنل به حالت کاربر منتقل شود. (S۳) این سوئیچ دوم نیز نیاز به تغییر نگاشت MMU و پاک‌سازی TLB دارد.

به جای استفاده از چنین فرایندهایی، یک برنامه همچنین می‌تواند طوری طراحی شود که بخش‌های مختلف آن توسط رشته‌های جداگانه اجرا شوند. در این صورت، ارتباط بین آن بخش‌ها عمدتاً از طریق داده‌های اشتراکی انجام می‌شود. گاهی این تعویض‌ها می‌تواند به‌طور کامل در فضای کاربر انجام شود،

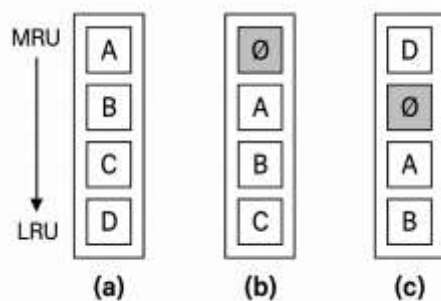
اگرچه در برخی پیاده‌سازی‌های دیگر، هسته سیستم‌عامل از وجود رشته‌ها آگاه است و آن‌ها را زمان‌بندی می‌کند. این موضوع می‌تواند به بهبود چشمگیر عملکرد منجر شود.

در نهایت، یک دلیل کاملاً مهندسی نرم‌افزار برای استفاده از رشته‌ها وجود دارد:

بسیاری از برنامه‌ها به سادگی آسان‌تر می‌شود اگر به صورت مجموعه‌ای از برنامه‌های همکار ساختار بندی شوند. به برنامه‌هایی فکر کنید که باید چندین وظیفه (کم‌وبیش مستقل) را به طور هم‌زمان انجام دهند، مانند همان مثال صفحه گسترده‌ای (spreadsheet) که قبلاً بررسی شد.

نکته ۳.۱ (پیشرفته: هزینه یک تعویض زمینه) مطالعات زیادی در زمینه اندازه‌گیری تأثیرات عملکردی تعویض زمینه‌ها انجام شده است. همانند بسیاری از موارد دیگر در سیستم‌های کامپیوتری، یافتن واقعیت دقیق کار آسانی نیست [Tshib ۲۰۰۷]. خاطرنشان می‌کند که رسیدگی به تیک‌های ساعت (clock ticks) به امری رایج در سیستم‌عامل‌ها تبدیل شده است، که این موضوع آن را به نامزدی عالی برای اندازه‌گیری سربار تبدیل می‌کند. یک هندلر ساعت هر **T میلی ثانیه** توسط یک وقفه‌ی ساعت فعال می‌شود. مقادیر معمول برای T بین **۰.۵ تا ۲۰ میلی ثانیه** است، که متناظر با فرکانس‌های وقفه بین **۲۰۰۰ هرتز تا ۵۰ هرتز** می‌باشد. این هندلر معمولاً در اجرای توابع زمان‌بندی و سرویس‌های CPU نقش دارد، سیگنال‌های هشدار ارسال می‌کند و به اجرای وظایف پس‌زمینه‌ای مانند اشتراک CPU کمک می‌کند. با تغییر ساده در فرکانسی که سخت‌افزار وقفه‌ها را تولید می‌کند، می‌توان تأثیری از سربار ناشی از وقفه را به دست آورد. برای اندازه‌گیری تأثیر عملکردی یک وقفه، تمایزی میان سربار مستقیم و غیرمستقیم وجود دارد. سربار مستقیم، مدت‌زمانی است که برای رسیدگی به تعویض واقعی زمینه صرف می‌شود، همراه با زمان لازم برای کار هندلر و بازگشت به کدی که وقفه را قطع کرده بود. سربار غیرمستقیم، شامل همه چیز دیگر است و عمدتاً ناشی از اختلالات کش (cache perturbation) است.

برای پردازنده‌های خطی مختلف، [Tshib ۲۰۰۷] دریافت که زمان تعویض زمینه در حدود **۰.۵ تا ۱ میکروثانیه** است و خود هندلر بسته به پیاده‌سازی، بین **۰.۵ تا ۷ میکروثانیه** زمان نیاز دارد.



شکل ۳.۲: سازمان‌دهی کش هنگام مواجهه با وقفه‌ها: (a) پیش از تعویض متن، (b) پس از تعویض متن، و (c) پس از دسترسی به بلوک D. (Liu و [Solihin ۲۰۱۰])

با این حال، مشخص می‌شود که سربار مستقیم واقعاً تأثیرگذار نیست. یک مطالعه تکمیلی توسط لیو و سولیهین [۲۰۱۰] نشان می‌دهد که تعویض متن (context switch) می‌تواند اختلال زیادی در کش ایجاد کند، که منجر به کاهش کارایی نسبت به وضعیت پیش از وقوع وقفه می‌شود. در واقع، برای حالت ساده‌ای از وقفه‌ها، tsafirir [۲۰۰۷] سربار غیرمستقیمی تقریباً ۸۰٪ را اندازه‌گیری کرده است.

برای درک آنچه در جریان است، سازمان‌دهی داده‌ها را همان‌طور که در شکل ۳.۲ نشان داده شده است، در نظر بگیرید. فرض کنید کش به گونه‌ای سازمان‌دهی شده است که هرگاه فضای کافی برای بارگذاری یک بلوک داده تازه نیاز باشد، یک بلوک داده‌ای که اخیراً کمتر استفاده شده است از کش حذف می‌شود.

شکل ۳.۲ (a) وضعیت را پیش از وقوع وقفه نشان می‌دهد. پس از رسیدگی به وقفه، ممکن است بلوک D از کش حذف شده باشد، و در نتیجه حفره‌ای مانند آنچه در شکل ۳.۲ (b) دیده می‌شود، باقی بماند. دسترسی به بلوک D باعث می‌شود که آن دوباره به کش بازگردانده شود، که ممکن است منجر به حذف بلوک C شود، و این روند ادامه می‌یابد. به بیان دیگر، حتی یک وقفه ساده نیز می‌تواند باعث یک سازمان‌دهی مجدد قابل توجه و نسبتاً پایدار در کش شود، که به‌نوبه‌ی خود بر عملکرد کلی یک برنامه تأثیر می‌گذارد.

```
۱. from multiprocessing import Process
۲. from time import *
۳. from random import *
۴.
۵. def sleeper(name):
۶.     t = gmtime()
۷.     s = randint(۱, ۲۰)
۸.     txt = str(t.tm_min) + ':' + str(t.tm_sec) + ' ' + name + ' is going to sleep for ' + str(s) + ' seconds'
۹.     print(txt)
۱۰.     sleep(s)
۱۱.     t = gmtime()
۱۲.     txt = str(t.tm_min) + ':' + str(t.tm_sec) + ' ' + name + ' has woken up'
۱۳.     print(txt)
۱۴.
۱۵. if __name__ == '__main__':
۱۶.     p = Process(target=sleeper, args=('eve',))
۱۷.     q = Process(target=sleeper, args=('bob',))
۱۸.     p.start(); q.start()
۱۹.     p.join(); q.join()
```

(الف)

۴۰:۲۳ eve قصد دارد به مدت ۱۴ ثانیه بخوابد

۴۰:۲۳ bob قصد دارد به مدت ۴ ثانیه بخوابد

۴۰:۲۷ bob بیدار شده است

۴۰:۳۷ eve بیدار شده است

(ب)

شکل ۳.۳: (الف) یک مثال ساده که در آن دو پردازش شروع شده‌اند، و (ب) خروجی پس از اجرا.

یک مثال ساده در پایتون.

برای ملموس تر کردن موضوع، بیایید به یک مثال ساده در پایتون نگاه کنیم، همچنین برای نشان دادن تفاوت‌های بین پردازش‌ها و رشته‌ها (threads) کد نشان داده شده در شکل ۳.۳(الف) را در نظر بگیرید که نشان می‌دهد چگونه شروع به جدا کردن پردازش‌ها در پایتون با استفاده از بسته multiprocessing می‌کنیم. هسته این مثال توسط تابع sleeping شکل گرفته است که به سادگی فرآیندی که آن را فراخوانی کرده، به مدت تعداد ثانیه‌های انتخاب شده به صورت تصادفی، به خواب می‌برد.

```
۱. from multiprocessing import Process
۲. from threading import Thread
۳.
۴. shared_x = random.randint(۱۰, ۹۹)
۵.
۶. def sleeping(name):
۷.     global shared_x
۸.     s = random.randint(۱, ۲۰)
۹.     sleep(s)
۱۰.     shared_x = shared_x + ۱
۱۱.
۱۲. def sleeper(name):
۱۳.     sleeplist = list()
۱۴.     for i in range(۳):
۱۵.         subsleeper = Thread(target=sleeping, args=(name + ' ' + str(i),))
۱۶.         sleeplist.append(subsleeper)
۱۷.
۱۸.     for s in sleeplist: s.start()
۱۹.     for s in sleeplist: s.join()
```

(الف)

eve مقدار shared_x را ۷۱ می‌بیند
۵۳:۲۱ eve ۰ قصد دارد به مدت ۲۰ ثانیه بخوابد
bob مقدار shared_x را ۸۴ می‌بیند
۵۳:۲۱ eve ۱ قصد دارد به مدت ۱۵ ثانیه بخوابد
۵۳:۲۱ eve ۲ قصد دارد به مدت ۳ ثانیه بخوابد
۵۳:۲۱ bob ۰ قصد دارد به مدت ۸ ثانیه بخوابد
۵۳:۲۱ bob ۱ قصد دارد به مدت ۱۶ ثانیه بخوابد
۵۳:۲۱ bob ۲ قصد دارد به مدت ۸ ثانیه بخوابد
۵۳:۲۴ eve ۲ بیدار شده است، و مقدار shared_x را ۷۲ می‌بیند
۵۳:۲۹ bob ۰ بیدار شده است، و مقدار shared_x را ۸۵ می‌بیند
۵۳:۲۹ bob ۲ بیدار شده است، و مقدار shared_x را ۸۶ می‌بیند
۵۳:۳۶ eve ۱ بیدار شده است، و مقدار shared_x را ۷۳ می‌بیند
۵۳:۳۷ bob ۱ بیدار شده است، و مقدار shared_x را ۸۷ می‌بیند
bob مقدار shared_x را ۸۷ می‌بیند

۵۳:۴۱ eve • بیدار شده است، و مقدار shared_x را ۷۴ می‌بیند

eve مقدار shared_x را ۷۴ می‌بیند

(ب)

شکل ۳.۴: (الف) یک مثال چندنخی (multithreading) که در آن دو پردازش شروع شده‌اند، هر کدام با سه نخ، و (ب) خروجی پس از اجرا.

برای ایجاد دو فرایند، عملیات Process را در خطوط ۱۶ و ۱۷ فراخوانی می‌کنیم تا هر کدام را به‌صورت جداگانه راه‌اندازی کنیم. عملیات join(اضافه شدن) به فرایند اصلی می‌گوید که منتظر بماند تا فرایندهای تازه ایجادشده کامل شوند. یک خروجی احتمالی در شکل ۳.۳ (b) نشان داده شده است که زمان را در قالب «دقیقه:ثانیه» نشان می‌دهد، زمانی که هر فرایند مقداری متن را چاپ می‌کند.

تفاوت بین رشته‌ها (threads) و فرایندها (processes) زمانی بهتر مشاهده می‌شود که مثال خود را گسترش می‌دهیم، همان‌طور که در شکل ۳.۴ آمده است (در اینجا بسیاری از دستورات مربوط به ثبت زمان و چاپ متن را حذف کرده‌ایم). در این حالت، بار دیگر دو فرایند با نام‌های eve و bob را راه‌اندازی می‌کنیم (کد مشابه خطوط ۱۹-۱۵ در شکل ۳.۳ است که برای وضوح بیشتر حذف شده است). هر فرایند به نوبه‌ی خود سه رشته را راه‌اندازی می‌کند که تابع sleeping را اجرا می‌کنند. تفاوت اصلی اینجا است که حالا یک متغیر مشترک به نام shared_x داریم. (برای ساده نگه داشتن موضوع، به اشتباه فرض می‌کنیم که تخصیص در خط ۱۰ یک عملیات اتمی است.) عملیات اتمی را در فصل ۵ با جزئیات توضیح خواهیم داد. چیزی که خروجی در شکل ۳.۴ نشان می‌دهد این است که این متغیر shared_x در میان رشته‌های یک فرایند مشترک است، اما بین دو فرایند eve و bob به اشتراک گذاشته نشده است. به عبارت دیگر، هر فرایند نمونه‌ی خودش از متغیر shared_x را دارد.

خروجی همچنین نشان می‌دهد که عملیات sleep در سطح رشته (Thread) و همچنین در سطح فرایند (Process) عمل می‌کند. در این حالت، هر رشته برای چند ثانیه به حالت تعلیق درمی‌آید، اما فرایندی که میزبان آن رشته است، هنگام فراخوانی sleep مسدود نمی‌شود. در عوض، یک رشته‌ی دیگر زمان‌بندی می‌شود (که آن هم sleep را فراخوانی می‌کند). (این که این عملیات چگونه پیاده‌سازی شده، برای برنامه‌نویس شفاف نیست (پنهان شده است). همچنین میزان اجرای رشته‌های مختلف بر روی هسته‌های متفاوت، در صورت در دسترس بودن، نیز از دید برنامه‌نویس پنهان است. اگر ما به‌ازای هر فرایند ۱۰۰۰ رشته اجرا می‌کردیم، باز هم زمان‌بندی دقیق و درستی مشاهده می‌کردیم. با این حال، اگر فراخوانی sleep را با یک حلقه‌ی شلوغ (busy waiting loop) جایگزین کنیم، مانند:

```
c = y * ۲۶۰۰۰۰۰۰
for i in range(c):
    x = x + ۱,۰
```

می‌توانیم ببینیم که اجرای رشته‌ها (threads) ممکن است برای هر فرایند به‌طور کامل به‌صورت سریالی انجام شود، در حالی که هر فرایند به یک هسته‌ی جداگانه اختصاص می‌یابد و در نتیجه به‌صورت موازی اجرا می‌شود. این که دقیقاً چه اتفاقی می‌افتد، بستگی به سیستم‌عامل زیرین و سامانه‌ی اجرایی پایتون دارد. در پیاده‌سازی‌های استاندارد پایتون، تمام رشته‌ها در یک فرایند، فقط به یک هسته اختصاص داده می‌شوند.

پیاده‌سازی رشته (Thread Implementation)

رشته‌ها اغلب در قالب یک بسته‌ی رشته ارائه می‌شوند. چنین بسته‌ای شامل عملیات‌هایی برای ایجاد و حذف رشته‌ها، همچنین عملیات‌هایی روی متغیرهای هم‌زمان‌سازی مانند mutex ها و متغیرهای شرطی می‌باشد. اساساً دو رویکرد برای پیاده‌سازی یک بسته‌ی رشته وجود دارد. رویکرد نخست ساخت یک کتابخانه‌ی رشته است که به‌طور کامل در فضای کاربر اجرا می‌شود. رویکرد دوم این است که هسته از وجود رشته‌ها آگاه باشد و آن‌ها را زمان‌بندی کند.

یک کتابخانه‌ی رشته در سطح کاربر چندین مزیت دارد. نخست آن که ایجاد و حذف رشته‌ها ارزان است. از آنجا که تمام مدیریت رشته‌ها در فضای آدرس‌دهی کاربر نگهداری می‌شود، هزینه‌ی ایجاد یک رشته عمدتاً توسط هزینه‌ی تخصیص حافظه برای راه‌اندازی پشته‌ی رشته تعیین می‌شود. به‌طور مشابه، حذف یک رشته عمدتاً شامل آزادسازی حافظه‌ی پشته‌ای است که دیگر مورد استفاده نیست. هر دو عملیات ارزان هستند.

دومین مزیت استفاده از رشته‌های سطح کاربر این است که تعویض زمینه‌ی رشته (thread context switching) اغلب می‌تواند تنها با چند دستور انجام شود. اساساً فقط مقادیر ثبات‌های پردازنده (CPU registers) باید ذخیره شده و سپس با مقادیر قبلی رشته‌ای که قرار است به آن سوئیچ شود بارگذاری شوند. نیازی به تغییر نگاشت‌های حافظه، تخلیه‌ی TLB، حسابداری CPU و مواردی از این دست نیست. تعویض زمینه‌ی رشته زمانی انجام می‌شود که دو رشته نیاز به همگام‌سازی پیدا کنند، برای مثال هنگام ورود به بخشی از داده‌ی اشتراکی. با این حال، همان‌طور که در یادداشت ۳.۱ اشاره شد، بخش زیادی از سربار تعویض زمینه ناشی از به هم ریختن حافظه‌ی نهان (cache) است.

یکی از معایب عمده‌ی رشته‌های سطح کاربر، ناشی از استفاده از مدل رشته‌سازی چند-به-یک است: چندین رشته به یک واحد زمان‌بندی‌شدنی نگاشته می‌شوند. ما این موضوع را پیش‌تر در مثال برنامه‌ی پایتون چندرشته‌ای ساده‌مان در شکل ۳.۴ مشاهده کردیم. در نتیجه، فراخوانی یک فراخوان انسدادی (blocking system call) بلافاصله باعث انسداد کل فرایندی می‌شود که رشته به آن تعلق دارد، و همچنین باعث انسداد تمام رشته‌های دیگر در آن فرایند نیز خواهد شد. همان‌طور که توضیح دادیم، رشته‌ها به‌ویژه زمانی مفید هستند که بخواهیم یک برنامه‌ی بزرگ را به بخش‌هایی تقسیم کنیم که می‌توانند به‌طور منطقی هم‌زمان اجرا شوند. در این حالت، مسدود شدن در ورودی/خروجی (I/O) نباید مانع از اجرای بخش‌های دیگر در همین حین شود. برای چنین کاربردهایی، رشته‌های سطح کاربر بی‌فایده‌اند.

این مشکلات را می‌توان تا حد زیادی با پیاده‌سازی نخ‌ها در هسته سیستم‌عامل برطرف کرد، که منجر به چیزی می‌شود که به آن مدل نخ یک‌به‌یک (مدل نخ یک‌به‌یک) گفته می‌شود، مدلی که در آن هر نخ یک واحد قابل زمان‌بندی (واحد قابل زمان‌بندی) است. بهایی که باید پرداخت شود این است که هر عملیات مربوط به نخ (ایجاد، حذف، همگام‌سازی و غیره) باید توسط هسته انجام شود و نیاز به یک فراخوانی سیستمی (فراخوانی سیستمی) دارد.

تعویض زمینه نخ (تعویض زمینه نخ) اکنون ممکن است به اندازه تعویض زمینه فرایند (تعویض زمینه فرایند) پرهزینه باشد با این حال، از آنجا که عملکرد تعویض زمینه معمولاً تحت تأثیر استفاده ناکارآمد از حافظه پنهان (حافظه پنهان) است، و نه تفاوت بین مدل نخ چند-به-یک (چند به یک) یا یک‌به‌یک (یک به یک)، بسیاری از سیستم‌عامل‌ها اکنون مدل اخیر (مدل اخیر) را ارائه می‌دهند، حتی اگر تنها به خاطر سادگی آن باشد.

یادداشت ۳-۲ پیشرفته مدل نخ بندی چند به چند

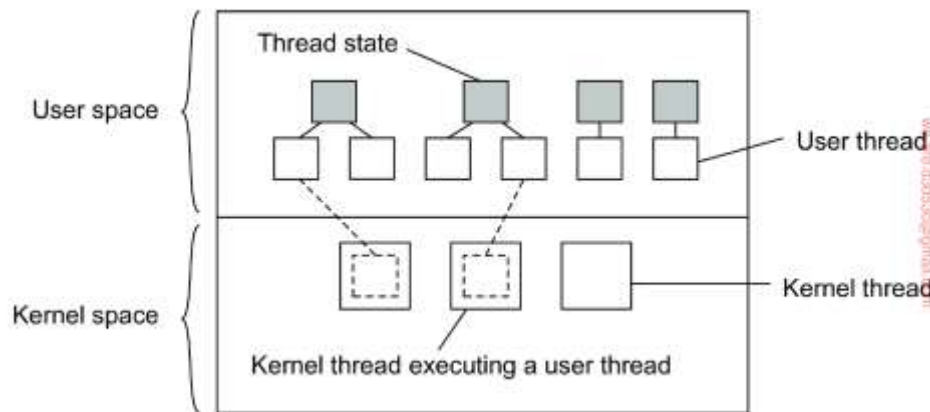
یک جایگزین برای دو مدل نخ بندی انتهایی یک به یک و چند به یک مدل ترکیبی از نخ‌های سطح کاربر و نخ‌های سطح هسته است که به آن مدل نخ بندی چند به چند گفته می‌شود در ادامه برای ساده تر شدن اصطلاحات از نخ‌های کاربر و نخ‌های سطح هسته استفاده می‌کنیم یک نخ هسته در زمینه (زمینه) یک فرایند اجرا می‌شود و می‌تواند چند نخ هسته برای هر فرایند وجود داشته باشد علاوه بر مدیریت نخ‌های هسته یک سیستم زمان اجرا (سیستم زمان اجرا) بسته ای برای نخ‌های سطح کاربر ارائه می‌دهد که عملیات معمولی مانند ایجاد و حذف نخ‌ها را برای برنامه‌ها فراهم می‌کند همچنین این بسته امکاناتی برای همگام‌سازی نخ‌ها مثل قفل‌های متقابل (قفل متقابل) و متغیرهای شرطی (متغیر شرطی) ارائه می‌دهد نکته مهم این است که بسته نخ به طور کامل در فضای کاربر پیاده سازی شده است و تمامی عملیات روی نخ‌ها بدون دخالت هسته انجام می‌شود.

این بسته نخ می‌تواند توسط چند نخ هسته به اشتراک گذاشته شود همان‌طور که در شکل ۳ ۵ نشان داده شده است این یعنی هر نخ هسته می‌تواند نخ سطح کاربر خودش را اجرا کند برنامه‌های چندنخی با ایجاد نخ‌های کاربر و نخ‌های سطح هسته ساخته می‌شوند و سپس هر نخ کاربر به یک نخ هسته اختصاص داده می‌شود.

ترکیب نخ‌های کاربر و هسته به صورت زیر کار می‌کند بسته نخ یک روال واحد برای زمان بندی نخ بعدی دارد هنگام ایجاد نخ هسته که از طریق یک فراخوانی سیستمی انجام می‌شود به نخ هسته یک پشته (پشته) اختصاص داده می‌شود و دستور داده می‌شود تا روال زمان بندی را اجرا کند و به دنبال نخ کاربری باشد که اجرا شود اگر چند نخ هسته وجود داشته باشد هر کدام زمان بند خود را اجرا می‌کنند.

جدول نخ‌ها که برای دنبال کردن مجموعه فعلی نخ‌ها استفاده می‌شود توسط نخ‌های هسته به اشتراک گذاشته می‌شود محافظت از این جدول برای تضمین دسترسی انحصاری متقابل (دسترسی انحصاری متقابل) از طریق قفل‌های متقابل (قفل متقابل) که به طور کامل در فضای کاربر

پیاده سازی شده اند انجام می شود به عبارت دیگر همگام سازی بین نخ های هسته نیازی به حمایت هسته ندارد جدول نخ ها معمولاً به صورت یک صف آماده (صف آماده) پیاده سازی می شود.



شکل ۳-۵ ترکیب نخ های سطح هسته و سطح کاربر

وقتی یک نخ هسته یک کاربر قابل اجرا پیدا می کند، زمینه اجرایی را به آن نخ تغییر می دهد در همین حال نخ های هسته دیگر ممکن است در حال جستجوی نخ های کاربر قابل اجرای دیگری باشند اگر یک نخ کاربر نیاز به مسدود شدن روی یک قفل متقابل (قفل متقابل) یا متغیر شرطی (متغیر شرطی) داشته باشد، مدیریت لازم را انجام می دهد و در نهایت روال زمان بندی را فراخوانی می کند وقتی نخ کاربر قابل اجرای دیگری پیدا شد، تعویض زمینه به آن نخ انجام می شود زیبایی این امر این است که نخ هسته ای که نخ کاربر را اجرا می کند نیازی به اطلاع ندارد تعویض زمینه به طور کامل در فضای کاربر پیاده سازی شده و برای نخ هسته مانند کد برنامه عادی به نظر می رسد.

حال ببینیم وقتی یک نخ کاربر یک فراخوانی سیستمی مسدودکننده انجام می دهد چه اتفاقی می افتد در این حالت اجرا از حالت کاربر به حالت هسته تغییر می کند اما همچنان در زمینه (زمینه) نخ هسته جاری ادامه می یابد در نقطه ای که نخ هسته جاری نمی تواند ادامه دهد سیستم عامل ممکن است تصمیم بگیرد زمینه اجرایی را به نخ هسته دیگری تغییر دهد که این نیز مستلزم انجام تعویض زمینه بازگشت به حالت کاربر است نخ هسته انتخاب شده به سادگی از جایی که قبلاً متوقف شده بود ادامه می دهد.

مزایای متعددی برای استفاده از نخ های هسته ای همراه با بسته نخ سطح کاربر وجود دارد اول اینکه ایجاد، حذف و همگام سازی نخ ها نسبتاً کم هزینه است و هیچ دخالتی از طرف هسته لازم نیست دوم اینکه به شرطی که یک فرایند نخ های هسته کافی داشته باشد یک فراخوانی سیستمی مسدودکننده کل فرایند را متوقف نمی کند سوم اینکه برنامه نیازی ندارد درباره نخ های هسته اطلاعی داشته باشد و تنها نخ های کاربر را می بیند چهارم نخ های هسته به راحتی می توانند در محیط های چندپردازشی با اجرای نخ های هسته مختلف روی پردازنده های مختلف یا هسته های مختلف استفاده شوند این چندپردازشی کاملاً از برنامه پنهان می ماند.

رویکردی که بیان شد در واقع شکل کلی ترکیب نخ های کاربر و هسته است این رویکرد در زبان برنامه نویسی گو (Go) پیاده سازی شده است [Donovan and Kernighan, ۲۰۱۵] سیستم زمان اجرای ۱۱ bfibre که توسط Karsten و Barghi در سال ۲۰۲۰ شرح و ارزیابی شده نیز نمونه ای برجسته برای مدل نخ بندی چند-به-چند است رویکردی کمی متفاوت در [Arachne [Qin et al., ۲۰۱۸] دیده می شود Arachne نخ های هسته را از برنامه ها مخفی می کند اما فرض می کند برنامه می تواند به طور کامل از هسته هایی که اختصاص داده شده اند مطلع باشد برای هر هسته اختصاص یافته یک نخ هسته اختصاص می دهد یک نتیجه مهم این است که Arachne از فراخوانی های ورودی/خروجی مسدودکننده پشتیبانی نمی کند.

به عنوان نکته پایانی، مهم است که به یاد داشته باشیم استفاده از نخ‌ها یکی از روش‌های سازماندهی اجرای هم‌زمان و موازی در یک برنامه است در عمل، اغلب می‌بینیم که برنامه‌ها به صورت مجموعه‌ای از فرایندهای هم‌زمان طراحی می‌شوند که به طور مشترک از امکانات ارتباط بین فرایندی که سیستم‌عامل فراهم می‌کند استفاده می‌کنند (نگاه کنید به رایبیز و رایبیز ۲۰۰۳ و استیونز ۱۹۹۹) نمونه خوبی از این رویکرد، ساختار سرور وب آپاچی است که به طور پیش‌فرض با چند فرایند برای رسیدگی به درخواست‌های ورودی آغاز به کار می‌کند هر فرایند یک نمونه تکنی از سرور را تشکیل می‌دهد اما توانایی برقراری ارتباط با دیگر نمونه‌ها از طریق روش‌های استاندارد را دارد.

همان‌طور که سینیواسان در سال ۲۰۱۰ استدلال کرده است، استفاده از فرایندها به جای نخ‌ها یک مزیت مهم دارد و آن جداسازی فضای داده‌هاست هر فرایند روی بخش مخصوص به خود از داده‌ها کار می‌کند و از طریق سیستم‌عامل از تداخل دیگران محافظت می‌شود اهمیت این جداسازی نباید دست کم گرفته شود برنامه‌نویسی با نخ‌ها به خاطر دشواری زیاد در مدیریت هم‌زمانی دسترسی به داده‌های مشترک، بسیار پیچیده تلقی می‌شود اما در صورت استفاده از فرایندها، فضاهای داده‌ای در نهایت توسط سخت‌افزار محافظت می‌شوند اگر یک فرایند تلاش کند به داده‌ای خارج از حافظه اختصاص داده شده به خود دسترسی یابد، سخت‌افزار یک استثنا ایجاد می‌کند که سپس توسط سیستم‌عامل پردازش می‌شود در مقابل، هیچ پشتیبانی سخت‌افزاری برای نخ‌هایی که به طور هم‌زمان در یک فرایند عمل می‌کنند وجود ندارد.

۳.۱.۲ نخ‌ها در سیستم‌های توزیع‌شده

یکی از ویژگی‌های مهم نخ‌ها این است که آن‌ها می‌توانند روشی مناسب برای انجام فراخوانی‌های سیستمی مسدودکننده فراهم کنند، بدون آن‌که کل فرایندی که نخ در آن اجرا می‌شود را مسدود کنند (به شرطی که از مدل نخ‌بندی چند-به-یک استفاده نشده باشد) این ویژگی باعث می‌شود نخ‌ها در سیستم‌های توزیع‌شده بسیار جذاب باشند، زیرا بیان ارتباطات در قالب نگهداری چندین اتصال منطقی به‌صورت هم‌زمان را بسیار آسان تر می‌کند. برای روشن‌تر شدن این موضوع، نگاهی دقیق‌تر به مشتری‌ها و سرورهای چندنخی خواهیم انداخت.

مشتری‌های چندنخی

برای رسیدن به سطح بالایی از شفافیت توزیع، سیستم‌های توزیع‌شده‌ای که در شبکه‌های گسترده (WAN) فعالیت می‌کنند ممکن است نیاز داشته باشند زمان‌های طولانی انتشار پیام بین فرایندی را پنهان کنند تأخیر رفت و برگشت در یک شبکه گسترده به راحتی می‌تواند در حد چند صد میلی‌ثانیه یا حتی چند ثانیه باشد.

روش معمول برای پنهان‌سازی این تأخیرهای ارتباطی، آغاز ارتباط و سپس ادامه دادن به کار دیگری است نمونه رایج این وضعیت در مرورگرهای وب دیده می‌شود معمولاً یک سند وب از یک فایل اچ‌تی‌ام‌ال تشکیل شده است که متن ساده را به همراه مجموعه‌ای از تصاویر، آیکون‌ها و غیره شامل می‌شود برای دریافت هر عنصر از یک سند وب، مرورگر باید یک اتصال تی‌سی‌پی/آی‌پی برقرار کرده، داده‌های دریافتی را بخواند و آن‌ها را به بخش نمایشگر منتقل کند برقراری اتصال و خواندن داده‌ها ذاتاً عملیات مسدودکننده هستند هنگام کار با ارتباطات مسافت‌بلند، این مشکل وجود دارد که زمان مورد نیاز برای کامل شدن هر عملیات ممکن است طولانی باشد.

مرورگر وب معمولاً با دریافت صفحه اچ‌تی‌ام‌ال آغاز می‌کند و سپس آن را نمایش می‌دهد برای پنهان‌سازی تأخیرهای ارتباطی تا جای ممکن، برخی مرورگرها داده‌ها را در حالی که هنوز در حال دریافت هستند، شروع به نمایش می‌کنند در حالی که متن در اختیار کاربر قرار می‌گیرد و امکاناتی مانند پیمایش فعال می‌شود، مرورگر ادامه می‌دهد به دریافت سایر فایل‌هایی که صفحه را تشکیل می‌دهند مانند تصاویر این تصاویر به محض دریافت، نمایش داده می‌شوند بنابراین کاربر لازم نیست منتظر بماند تا همه اجزای صفحه دریافت شوند تا بتواند محتوای آن را ببیند.

در عمل مشاهده می‌شود که مرورگر وب در حال انجام چندین کار به‌صورت هم‌زمان است همان‌طور که مشخص است، توسعه مرورگر به‌عنوان یک مشتری چندنخی، مسائل را به‌طور قابل‌توجهی ساده‌تر می‌کند به محض این‌که فایل اصلی اچ‌تی‌ام‌ال دریافت شد، نخ‌های جداگانه‌ای می‌توانند فعال شوند تا وظیفه دریافت سایر بخش‌ها را بر عهده بگیرند هر نخ یک اتصال جداگانه با سرور برقرار می‌کند و داده‌ها را دریافت می‌کند برقراری اتصال و خواندن داده از سرور را می‌توان با استفاده از فراخوانی‌های سیستمی استاندارد (مسدودکننده) برنامه‌نویسی کرد، به شرطی که یک فراخوانی مسدودکننده کل فرایند را متوقف نکند همان‌طور که در [استیونز ۱۹۹۸] نیز نشان داده شده، کد مربوط به هر نخ یکسان و از همه مهم‌تر، ساده است.

در این میان، کاربر تنها تاخیرهایی در نمایش تصاویر و موارد مشابه را مشاهده می‌کند، اما می‌تواند بدون مشکل در سند مرور کند.

یک مزیت مهم دیگر در استفاده از مرورگرهای وب چندنخی این است که امکان باز کردن چندین اتصال به صورت همزمان وجود دارد در مثال قبلی، چند اتصال به یک سرور برقرار شد اگر آن سرور بسیار شلوغ باشد یا به طور کلی کند عمل کند، بهبود قابل توجهی در عملکرد نسبت به حالتی که فایل های صفحه به صورت پشت سرهم و یکی یکی دریافت می شوند مشاهده نخواهد شد.

با این حال، در بسیاری از موارد، سرورهای وب روی چندین ماشین تکرار (کپی) شده اند، به طوری که هر سرور مجموعه ای یکسان از اسناد وب را ارائه می دهد این سرورهای تکرار شده در یک محل قرار دارند و با یک نام یکسان شناخته می شوند.

وقتی درخواستی برای یک صفحه وب دریافت می شود، این درخواست به یکی از سرورها هدایت می شود، اغلب با استفاده از راهبرد چرخشی (round-robin) یا تکنیکی دیگر برای متعادل سازی بار در هنگام استفاده از یک مشتری چندنخی، ممکن است اتصال ها به نسخه های مختلف سرور برقرار شوند که اجازه می دهد داده ها به صورت موازی منتقل شوند در نتیجه، سند کامل وب در زمانی بسیار کوتاه تر نسبت به زمانی که تنها یک سرور (غیر تکراری) وجود دارد، نمایش داده می شود.

این روش تنها زمانی ممکن است که مشتری بتواند واقعاً جریان های موازی از داده های دریافتی را مدیریت کند نخ ها برای این منظور کاملاً مناسب هستند.

یادداشت ۳.۳ (پیشرفته: بهره گیری از نخ های سمت مشتری برای بهبود عملکرد)

اگرچه فرصت های آشکاری برای استفاده از نخ ها جهت دستیابی به عملکرد بالا وجود دارد، اما جالب است که ببینیم آیا واقعاً از چندنخی بودن به طور مؤثر استفاده می شود یا نه در یک مطالعه برای بررسی این که تا چه اندازه نخ های متعدد یک پردازنده چند هسته ای را به کار می گیرند، بلیک و همکارانش [۲۰۱۰] اجرای برنامه های مختلف را روی معماری های مدرن بررسی کردند مرورگرها، مانند بسیاری دیگر از برنامه های سمت مشتری، ذاتاً تعاملی هستند، به همین دلیل ممکن است زمان بیکار بودن پردازنده در آن ها نسبتاً زیاد باشد.

برای اندازه گیری دقیق میزان استفاده از یک پردازنده چند هسته ای، بلیک و همکارانش از معیاری به نام موازی سازی در سطح نخ (TLP) استفاده کردند فرض کنید C_i نشان دهنده کسری از زمان باشد که دقیقاً i نخ به طور همزمان در حال اجرا هستند در این صورت، موازی سازی در سطح نخ به صورت زیر تعریف می شود:

$$TLP = \frac{\sum_{i=1}^N i \cdot c_i}{1 - c_0}$$

که در آن N بیشینه تعداد نخ هایی است که می توانند به طور همزمان اجرا شوند

در مطالعه آن ها، یک مرورگر وب معمولی در آن زمان مقدار TLP بین ۱/۵ تا ۲/۵ داشت، به این معنا که برای بهره گیری مؤثر از موازی سازی، دستگاه سمت مشتری باید دارای دو یا سه هسته (یا به طور مشابه، دو تا سه پردازنده) باشد

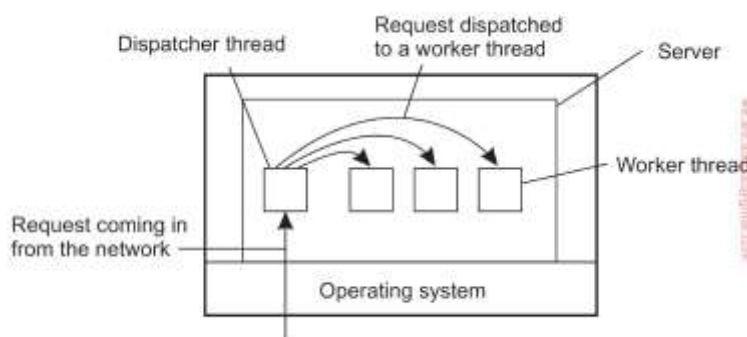
این نتایج زمانی جالب می شوند که بدانیم مرورگرهای وب مدرن صدها نخ ایجاد می کنند و ده ها نخ به طور همزمان فعال هستند (توجه داشته باشید که نخ فعال الزاماً در حال اجرا نیست؛ ممکن است منتظر تکمیل یک درخواست ورودی/خروجی مسدود شده باشد) بنابراین مشاهده می کنیم که چندنخی بودن بیشتر برای سازمان دهی یک برنامه مورد استفاده قرار می گیرد، اما این چندنخی بودن منجر به بهبود چشمگیر عملکرد از طریق بهره گیری از سخت افزار نمی شود.

این که مرورگرها می توانند برای بهره برداری از موازی سازی به طور مؤثر طراحی شوند، مثلاً در کار می پروویج و بادیک [۲۰۱۰] نشان داده شده است نویسندگان با سازگار کردن الگوریتم های موجود موفق شدند چندین برابر سرعت اجرای برنامه را افزایش دهند.

سرورهای چندنخی

اگرچه مشتریهای چندنخی مزایای مهمی دارند، اما استفاده اصلی از چندنخی بودن در سیستمهای توزیع شده معمولاً در سمت سرور صورت می گیرد تجربه نشان داده که چندنخی بودن نه تنها کدنویسی سرور را به طور قابل توجهی ساده تر می کند، بلکه توسعه سرورهایی که از موازی سازی برای دستیابی به عملکرد بالا استفاده می کنند را نیز بسیار آسان تر می سازد، حتی در سیستمهایی که تنها یک پردازنده دارند با این حال، با وجود پردازنده های چند هسته ای مدرن، استفاده از چندنخی بودن برای بهره گیری از موازی سازی کاملاً بدیهی و ضروری به نظر می رسد.

برای درک مزایای نخ ها در نوشتن کد سرور، به سازمان دهی یک سرور فایل توجه کنید که گاهی اوقات مجبور است منتظر دیسک بماند این سرور فایل معمولاً در انتظار دریافت یک درخواست عملیاتی از سمت مشتری است، سپس آن درخواست را اجرا کرده و در پایان پاسخ را ارسال می کند. یکی از روش های رایج و پرطرفدار در این زمینه، در شکل ۳/۶ نشان داده شده است در این ساختار، یک نخ به عنوان توزیع کننده (dispatcher)، درخواست های ورودی برای عملیات روی فایل ها را دریافت می کند این درخواست ها از سوی مشتریان به یک نقطه پایانی شناخته شده برای این سرور ارسال می شوند پس از بررسی درخواست، سرور یک نخ کارگر بیکار (یعنی نخ مسدود شده) را انتخاب کرده و درخواست را به آن واگذار می کند.



شکل ۳/۶: سرور چندنخی سازمان یافته در مدل توزیع کننده/کارگر

نخ کارگر با انجام یک عملیات خواندن مسدود کننده روی سیستم فایل محلی ادامه می دهد که ممکن است باعث تعلیق نخ تا زمان بازایی داده ها از دیسک شود اگر نخ تعلیق شود، نخ دیگری برای اجرا انتخاب می شود برای مثال ممکن است توزیع کننده انتخاب شود تا کارهای بیشتری دریافت کند یا نخ کارگر دیگری که اکنون آماده اجرا است انتخاب شود.

حال تصور کنید سرور فایل بدون نخ چگونه نوشته می شد یک امکان این است که سرور به صورت تکنخی عمل کند حلقه اصلی سرور فایل یک درخواست دریافت کرده، آن را بررسی کرده و تا پایان انجام می دهد قبل از آن که درخواست بعدی را دریافت کند در هنگام انتظار برای دیسک، سرور بیکار بوده و هیچ درخواست دیگری را پردازش نمی کند بنابراین درخواست های سایر مشتریان نمی توانند رسیدگی شوند علاوه بر این، اگر سرور فایل روی یک ماشین اختصاصی اجرا شود، که معمولاً همین طور است، پردازنده کاملاً بیکار می ماند در حالی که سرور فایل در انتظار دیسک است نتیجه نهایی این است که تعداد درخواست های پردازش شده در واحد زمان بسیار کمتر خواهد بود بنابراین نخ ها عملکرد قابل توجهی را به ارمغان می آورند اما هر نخ به صورت متوالی و به شیوه معمول برنامه نویسی می شود.

تا اینجا دو طراحی ممکن را دیده ایم سرور فایل چندنخی و سرور فایل تکنخی

گزینه سوم این است که سرور به عنوان یک ماشین حالت متناهی بزرگ تکنخی اجرا شود وقتی یک درخواست وارد می شود تنها نخ آن را بررسی می کند اگر درخواست از کش حافظه اصلی قابل پاسخگویی باشد که مشکلی نیست اما اگر نباشد نخ باید به دیسک دسترسی پیدا کند. با این حال به جای انجام عملیات مسدود کننده روی دیسک، نخ یک عملیات ناهمزمان (یعنی غیر مسدود کننده) روی دیسک برنامه ریزی می کند که بعداً توسط سیستم عامل قطع می شود.

برای اینکه این کار عملی شود، نخ وضعیت درخواست را ثبت می کند (یعنی اینکه یک عملیات دیسک معلق دارد) و به ادامه بررسی درخواست های ورودی دیگری که نیاز به توجه دارند می پردازد.

زمانی که عملیات معلق روی دیسک تکمیل شود، سیستم عامل نخ مربوطه را مطلع می کند که سپس در زمان مناسب وضعیت درخواست مرتبط را بررسی کرده و به پردازش آن ادامه می دهد در نهایت پاسخ به مشتری مبدا ارسال می شود که این بار نیز از یک فراخوانی غیرمسدودکننده برای ارسال پیام روی شبکه استفاده می شود.

در این طراحی مدل «فرآیند متوالی» که در دو حالت اول داشتیم از بین می رود هر بار که نخ نیاز به انجام یک عملیات مسدودکننده دارد باید دقیقاً جایی را که در پردازش درخواست بوده ثبت کند و ممکن است وضعیت اضافی دیگری نیز ذخیره کند پس از انجام این کار می تواند عملیات را شروع کرده و به کارهای دیگر پردازد کارهای دیگر یعنی پردازش درخواست های تازه رسیده یا پردازش پس از عملیات درخواست هایی که عملیاتشان قبلاً آغاز شده و به پایان رسیده است البته اگر کاری برای انجام دادن نباشد نخ ممکن است واقعاً مسدود شود در واقع ما رفتار چند نخ و پشته های مربوط به هر کدام را به روش سخت شبیه سازی می کنیم این فرآیند به صورت یک ماشین حالت متناهی عمل می کند که رویدادی دریافت کرده و سپس بسته به آنچه در رویداد است به آن واکنش نشان می دهد.

ویژگی ها	مدل
همزمانی، فراخوانی های سیستم مسدودکننده	چندنخی
بدون همزمانی، فراخوانی های سیستم مسدودکننده	فرآیند تکنخی
همزمانی، فراخوانی های سیستم غیرمسدودکننده	ماشین حالت متناهی

شکل ۳.۷: سه روش برای ساخت یک سرور

اکنون باید روشن باشد که نخها چه مزایایی دارند آنها امکان حفظ ایده ی فرآیندهای متوالی که فراخوانی های سیستم مسدودکننده انجام می دهند را فراهم می کنند و در عین حال همزمانی را نیز به دست می آورند فراخوانی های سیستم مسدودکننده برنامه نویسی را آسان تر می کنند چون مانند فراخوانی های معمولی توابع به نظر می رسند علاوه بر این، وجود نخهای متعدد اجازه همزمانی و در نتیجه بهبود عملکرد را می دهد سرور تکنخی سهولت و سادگی فراخوانی های سیستم مسدودکننده را حفظ می کند اما ممکن است به شدت عملکرد را از نظر تعداد درخواست هایی که در واحد زمان قابل پردازش هستند محدود کند روش ماشین حالت متناهی از طریق همزمانی به عملکرد بالایی دست می یابد اما از فراخوانی های غیرمسدودکننده استفاده می کند که به طور کلی برنامه نویسی آنها دشوار و نگهداری آنها سخت است این مدل ها در شکل ۳.۷ خلاصه شده اند.

۳.۲۱ مجازی سازی

باز هم توجه داشته باشید که به جای استفاده از نخها، می توانیم از چندین فرآیند برای سازماندهی یک سرور استفاده کنیم (که در این حالت در واقع یک سرور چندفرآیندی داریم) مزیت این کار این است که سیستم عامل می تواند محافظت بیشتری در برابر دسترسی تصادفی به داده های مشترک ارائه دهد با این حال، اگر فرآیندها نیاز به ارتباط زیاد داشته باشند، ممکن است اثر منفی قابل توجهی روی عملکرد نسبت به استفاده از نخها مشاهده شود.

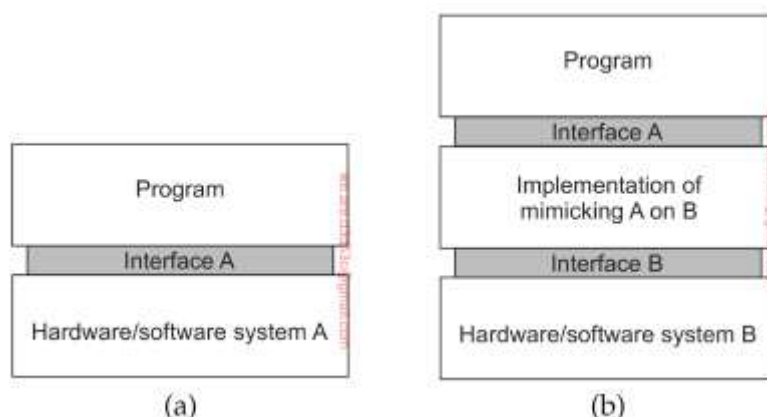
۳.۲۱ مجازی سازی

نخها و فرآیندها را می توان به عنوان راهی برای انجام چند کار به صورت همزمان در نظر گرفت در واقع، آنها به ما اجازه می دهند قطعاتی از برنامه ها را بسازیم که به نظر می رسد به طور همزمان اجرا می شوند روی یک کامپیوتر تک پردازنده (تک هسته) این اجرای همزمان البته توهمی بیش نیست چون فقط یک پردازنده وجود دارد و در هر لحظه فقط یک دستور از یک نخ یا فرآیند اجرا می شود با سوئیچ سریع بین نخها و فرآیندها توهم موازی سازی ایجاد می شود..

این جداسازی بین داشتن یک CPU و توانایی وانمود کردن به وجود چند CPU را می‌توان به منابع دیگر نیز گسترش داد که منجر به چیزی می‌شود که به آن مجازی‌سازی منابع گفته می‌شود این مجازی‌سازی سال‌هاست که استفاده می‌شود اما به دلیل رواج و پیچیدگی بیشتر سیستم‌های کامپیوتری (خصوصاً توزیع شده) دوباره مورد توجه قرار گرفته چون نرم‌افزارهای کاربردی اغلب از نرم‌افزارهای پایه و سخت‌افزارهای زیرین خود طول عمر بیشتری دارند.

مجازی‌سازی و سیستم‌های توزیع شده

یکی از مهم‌ترین دلایل معرفی مجازی‌سازی در دهه ۱۹۷۰، اجازه دادن به اجرای نرم‌افزارهای قدیمی روی سخت‌افزارهای گران‌قیمت مین‌فریم بود این نرم‌افزارها نه تنها شامل برنامه‌های کاربردی مختلف، بلکه سیستم‌عامل‌هایی بودند که برای آنها توسعه یافته بودند این روش برای پشتیبانی از نرم‌افزارهای قدیمی به طور موفقیت‌آمیزی روی مین‌فریم‌های (IBM ۳۷۰ و جانشینان آن) به کار گرفته شد که ماشین‌های مجازی ارائه می‌دادند و سیستم‌عامل‌های مختلف برای آنها پورت شده بود.



شکل ۳.۸: (a) سازماندهی کلی بین برنامه، واسط (رابط) و سیستم (b) سازماندهی کلی مجازی‌سازی سیستم A روی سیستم B

با ارزان‌تر شدن سخت‌افزار، قدرتمندتر شدن کامپیوترها، و کاهش تنوع سیستم‌عامل‌ها، مجازی‌سازی کمتر به یک مسئله تبدیل شد. اما شرایط از اواخر دهه ۱۹۹۰ دوباره تغییر کرده است. اول اینکه، در حالی که سخت‌افزار و نرم‌افزارهای سطح پایین به سرعت تغییر می‌کنند، نرم‌افزارهای سطح بالاتر (مثل میدل‌ویر و برنامه‌ها) معمولاً پایدارتر هستند. به عبارت دیگر، ما با وضعیتی مواجهیم که نرم‌افزارهای قدیمی (legacy) نمی‌توانند به همان سرعتی که پلتفرم‌های وابسته به آنها تغییر می‌کنند، به‌روزرسانی شوند. مجازی‌سازی می‌تواند با پورت کردن واسط‌های قدیمی به پلتفرم‌های جدید کمک کند و در نتیجه به سرعت پلتفرم‌های جدید را برای دسته‌های بزرگی از برنامه‌های موجود باز کند.

همچنین اهمیت دارد که شبکه‌ها کاملاً فراگیر شده‌اند. تصور اینکه کامپیوتر مدرن به شبکه متصل نباشد، سخت است. در عمل، این اتصال باعث می‌شود که مدیران سیستم مجموعه بزرگی از سرورها را نگهداری کنند که هر کدام برنامه‌های متفاوتی اجرا می‌کنند و توسط کلاینت‌ها قابل دسترسی هستند. در عین حال، منابع مختلف باید به آسانی در دسترس این برنامه‌ها باشند. مجازی‌سازی می‌تواند کمک بزرگی باشد؛ با کاهش تنوع پلتفرم‌ها و ماشین‌ها، به هر برنامه اجازه می‌دهد روی ماشین مجازی خود اجرا شود، شامل کتابخانه‌ها و سیستم‌عامل مربوطه، که همه روی یک پلتفرم مشترک اجرا می‌شوند.

این نوع آخر مجازی‌سازی درجه بالایی از قابلیت حمل و انعطاف‌پذیری را فراهم می‌کند. برای مثال، برای پیاده‌سازی شبکه‌های تحویل محتوا که بتوانند به آسانی از تکثیر محتوای پویا پشتیبانی کنند، [Awadallah و Rosenblum ۲۰۰۲] استدلال کرده‌اند که مدیریت بسیار ساده‌تر می‌شود اگر سرورهای لبه از مجازی‌سازی پشتیبانی کنند، که به این ترتیب یک سایت کامل، شامل محیط آن، قابل مدیریت است و محیط، به صورت پویا کپی شود. این دلایل

همچنان معتبر هستند و در واقع، قابلیت قابل حمل بودن (portability) شاید مهم‌ترین دلیلی باشد که چرا مجازی‌سازی نقش کلیدی در بسیاری از سامانه‌های توزیع شده ایفا می‌کند.

در نهایت، یکی از دلایل مهم استفاده از مجازی‌سازی این است که روشی اضافی برای ایزوله‌سازی کد فراهم می‌کند؛ موضوعی که به‌ویژه در رایانش ابری اهمیت دارد. با این حال، مجازی‌سازی در عین حال تهدیدات امنیتی جدیدی نیز به همراه می‌آورد

یادداشت ۳.۴ (بحث: نرم‌افزار پایدار؟)

اگرچه واقعاً نرم‌افزارهای قدیمی زیادی وجود دارند که می‌توانند از رابط‌های پایدار در برابر سخت‌افزارهای زیربنایی که به سرعت در حال تغییرند بهره‌مند شوند، اما این یک اشتباه است که تصور کنیم نرم‌افزارهایی که برای خدمات عمومی در دسترس هستند، به‌ندرت تغییر می‌کنند. با گرایش فزاینده به سمت رایانش سمت سرور، در قالب نرم‌افزار به‌عنوان سرویس (SaaS)، بسیاری از نرم‌افزارها می‌توانند برای یک بستر نسبتاً همگن که کاملاً در مالکیت سازمان ارائه‌دهنده‌ی آن سرویس است، نگهداری شوند. در نتیجه، نگهداری از محصولات نرم‌افزاری بسیار آسان‌تر می‌شود، چرا که دیگر نیازی به توزیع تغییرات بین میلیون‌ها مشتری بالقوه وجود ندارد. در واقع، تغییرات ممکن است خیلی سریع یکی پس از دیگری رخ دهند هم راستا با تغییرات در سخت‌افزار و پلتفرم اما بدون آنکه هیچ کاربری حتی متوجه قطعی سرویس شود [Barroso و همکاران، ۲۰۱۸]

انواع مجازی‌سازی

راه‌های مختلفی برای پیاده‌سازی مجازی‌سازی وجود دارد. یک مرور کلی از این رویکردهای گوناگون توسط اسمیت و نایر [۲۰۰۵a] ارائه شده است. گزارش جدیدتری نیز توسط بوگنیون و همکاران [۲۰۱۷] منتشر شده که جزئیات فنی زیادی درباره‌ی نحوه‌ی پیاده‌سازی انواع مختلف مجازی‌سازی ارائه می‌دهد. برای درک تفاوت‌های موجود در مجازی‌سازی، مهم است بدانیم که سیستم‌های کامپیوتری معمولاً چهار نوع مختلف از رابط‌ها را در سه سطح مختلف ارائه می‌دهند:

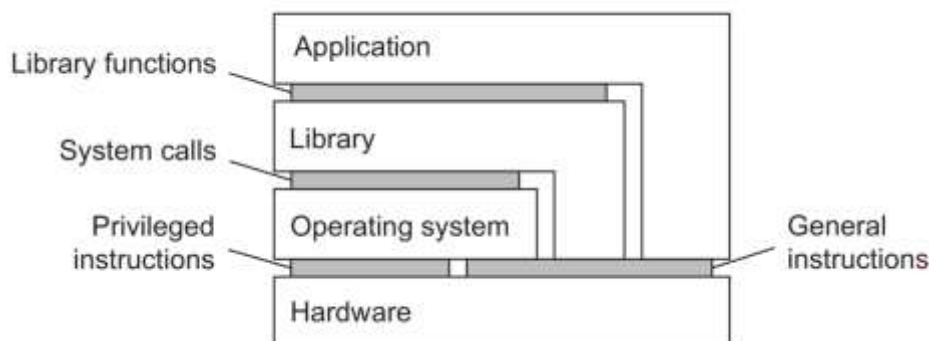
۱. رابطی بین سخت‌افزار و نرم‌افزار که به آن معماری مجموعه دستورالعمل‌ها (ISA – Instruction Set Architecture) گفته می‌شود، و شامل مجموعه‌ای از دستورالعمل‌های ماشین است. این مجموعه به دو زیرمجموعه تقسیم می‌شود:

- دستورالعمل‌های سطح بالا (Privileged instructions)، که تنها توسط سیستم‌عامل مجاز به اجرا هستند.
- دستورالعمل‌های عمومی (General instructions)، دستوراتی که توسط هر برنامه‌ای قابل اجرا هستند.

۲. یک رابط متشکل از فراخوانی‌های سیستمی که توسط سیستم‌عامل ارائه می‌شود

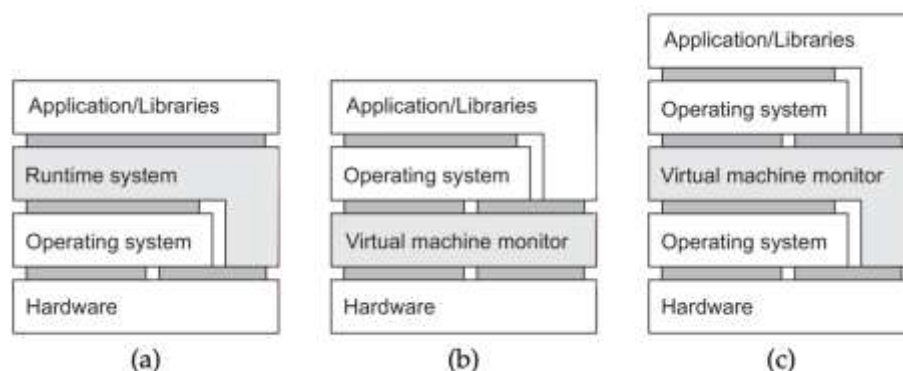
۳. یک رابط شامل فراخوانی‌های کتابخانه‌ای که معمولاً آنچه به عنوان رابط برنامه‌نویسی کاربردی (API) شناخته می‌شود را شکل می‌دهد. اغلب، فراخوانی‌های سیستمی ذکر شده قبلی توسط API پنهان می‌شوند.

این انواع مختلف در شکل ۳.۹ نشان داده شده‌اند. جوهره‌ی مجازی‌سازی، شبیه‌سازی رفتار این رابط‌ها است.



شکل ۳.۹: رابط‌های مختلف ارائه شده توسط سیستم‌های کامپیوتری.

مجازی‌سازی می‌تواند به دو روش مختلف انجام شود. اول اینکه می‌توان یک سیستم زمان اجرا ساخت که در اصل یک مجموعه دستورالعمل انتزاعی ارائه می‌دهد تا برای اجرای برنامه‌ها استفاده شود. دستورالعمل‌ها می‌توانند تفسیر شوند (مانند محیط زمان اجرای جاوا)، اما همچنین می‌توانند شبیه‌سازی شوند، همان‌طور که برای اجرای برنامه‌های ویندوز روی پلتفرم‌های یونیکس انجام می‌شود. توجه داشته باشید که در حالت دوم، شبیه‌ساز همچنین باید رفتار فراخوانی‌های سیستمی را نیز شبیه‌سازی کند، که به طور کلی کاری پیچیده و دشوار است. این نوع مجازی‌سازی، که در شکل ۳.۱۰ (a) نشان داده شده است، منجر به چیزی می‌شود که اسمیت و نایر [۲۰۰۵a] آن را ماشین مجازی فرایند (process virtual machine) می‌نامند و تأکید می‌کنند که مجازی‌سازی فقط برای یک فرایند واحد است.



شکل ۳.۱۰: (a) یک ماشین مجازی فرایند، (b) یک مانیتور ماشین مجازی بومی، (c) یک مانیتور ماشین مجازی میزبانی شده.

روش جایگزین برای مجازی‌سازی، که در شکل ۳.۱۰ (b) نشان داده شده، ارائه سیستمی است که به صورت یک لایه عمل می‌کند و سخت‌افزار اصلی را محافظت می‌کند، اما مجموعه کامل دستورالعمل‌های همان سخت‌افزار (یا سخت‌افزار دیگر) را به عنوان یک رابط ارائه می‌دهد. این روش منجر به چیزی می‌شود که به عنوان مانیتور ماشین مجازی بومی (native virtual machine monitor) شناخته می‌شود. این نام به این دلیل است که مستقیماً بر روی سخت‌افزار زیربنایی پیاده‌سازی شده است. توجه داشته باشید که رابط ارائه شده توسط یک مانیتور ماشین مجازی می‌تواند به طور همزمان به چندین برنامه ارائه شود. در نتیجه، اکنون امکان دارد چندین سیستم‌عامل مهمان مختلف به صورت مستقل و همزمان روی یک پلتفرم واحد اجرا شوند.

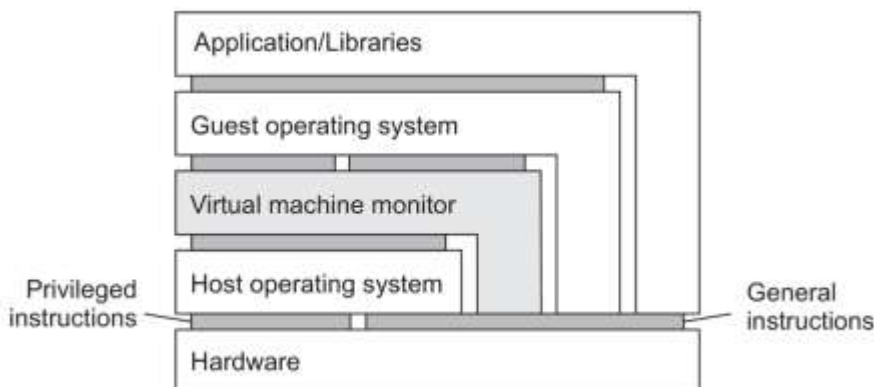
یک مانیتور ماشین مجازی بومی باید دسترسی به منابع مختلف مانند حافظه‌های ذخیره سازی خارجی و شبکه‌ها را فراهم و مدیریت کند. مانند هر سیستم‌عامل دیگری، این یعنی باید درایورهای دستگاه برای این منابع را پیاده‌سازی کند. به جای انجام این همه کار از ابتدا، یک مانیتور ماشین مجازی میزبانی شده (hosted virtual machine monitor) بر روی یک سیستم‌عامل میزبان قابل اعتماد اجرا می‌شود، همان‌طور که در شکل ۳.۱۰ (C) نشان داده شده است. در این حالت، مانیتور ماشین مجازی می‌تواند از امکانات موجود که توسط سیستم‌عامل میزبان فراهم شده‌اند، استفاده کند. معمولاً باید به آن امتیازات ویژه داده شود و نمی‌تواند صرفاً به عنوان یک برنامه سطح کاربر اجرا شود. استفاده از مانیتور ماشین مجازی میزبانی شده در سامانه‌های توزیع‌شده مدرن مانند مراکز داده و رایانش ابری بسیار رایج است.

همان‌طور که توسط روزنبلوم و گارفینکل [۲۰۰۵] مطرح شده، ماشین‌های مجازی در زمینه قابلیت اطمینان و امنیت سیستم‌های (توزیع‌شده) اهمیت زیادی دارند. از آنجا که این ماشین‌ها امکان ایزوله کردن کامل یک برنامه و محیط آن را فراهم می‌کنند، خطا یا حمله امنیتی دیگر لازم نیست کل ماشین را تحت تأثیر قرار دهد. علاوه بر این، همان‌طور که قبلاً هم اشاره کردیم، قابلیت جابجایی (portability) به‌طور قابل توجهی بهبود می‌یابد، زیرا ماشین‌های مجازی باعث جدا شدن بیشتر سخت‌افزار و نرم‌افزار می‌شوند و اجازه می‌دهند یک محیط کامل از یک ماشین به ماشین دیگر منتقل شود. در بخش ۳.۵ دوباره به موضوع مهاجرت (migration) باز می‌گردیم. یادداشت ۳.۵ (پیشرفته: درباره عملکرد ماشین‌های مجازی)

ماشین‌های مجازی عملکردی شگفت‌انگیز دارند. در واقع، بسیاری از مطالعات نشان می‌دهند که ماشین‌های مجازی مدرن عملکردی نزدیک به اجرای مستقیم برنامه‌ها روی سیستم‌عامل میزبان دارند. بیایید نگاهی دقیق‌تر به عملکرد داخلی ماشین‌های مجازی بیندازیم. شرحی دقیق و جامع درباره ماشین‌های مجازی توسط اسمیت و نایر [۲۰۰۵b] ارائه شده است.

بخشی از پاسخ به مسائل عملکرد در شکل ۳.۱۱ نشان داده شده است که گسترشی از شکل ۳.۱۰ (C) به حساب می‌آید: بخش بزرگی از کدی که یک مانیتور ماشین مجازی، سیستم‌عامل مهمان و برنامه را تشکیل می‌دهد، به‌صورت بومی روی سخت‌افزار زیربنایی اجرا می‌شود. به‌ویژه، تمامی دستورالعمل‌های عمومی (یعنی غیر ویژه) ماشین مستقیماً توسط سخت‌افزار زیربنایی اجرا می‌شوند.

این رویکرد جدید نیست و بر اساس پژوهشی از پوپک و گلدبرگ [۱۹۷۴] بنا شده است که الزامات اجرای بهینه ماشین‌های مجازی را رسمی کردند. به طور خلاصه، پوپک و گلدبرگ فرض کردند که ماشین زیربنایی حداقل دو حالت عملیاتی (حالت سیستم و حالت کاربر) دارد، که یک زیرمجموعه از دستورالعمل‌ها فقط در حالت سیستم قابل اجرا هستند، و آدرس‌دهی حافظه نسبی است (یعنی یک آدرس فیزیکی با افزودن یک آدرس نسبی به آفستی که در یک رجیستر جابجایی ذخیره شده به دست می‌آید). همچنین تمایزی بین دو نوع دستورالعمل قائل شدند: دستورالعمل ویژه (privileged instruction) دستوری که اگر و تنها اگر در حالت کاربر اجرا شود، باعث بروز یک تله (trap) به سیستم‌عامل می‌شود. دستورالعمل غیر ویژه (nonprivileged instruction) سایر دستورالعمل‌ها.



شکل ۳.۱۱: برنامه‌ها، سیستم‌عامل مهمان، مانیتور ماشین مجازی و سیستم‌عامل میزبان روی یک پلتفرم سخت‌افزاری واحد.

با توجه به این فرضیات رسمی، پوپک و گلدبرگ دو دسته از دستورالعمل‌های ویژه را تعریف کردند، دستورالعمل حساس به کنترل (control-sensitive instruction): دستوری که ممکن است پیکربندی ماشین را تغییر دهد. مثال معمول آن، دستوری است که روی نحوه چیدمان حافظه تأثیر می‌گذارد، مثلاً با تغییر آفست حافظه که در رجیستر جابجایی ذخیره شده است. مثال دیگر، دستوراتی هستند که جدول وقفه‌ها را تحت تأثیر قرار می‌دهند، جدولی که اشاره‌گرهایی به پردازش‌گرهای وقفه (interrupt handlers) دارد.

دستورالعمل حساس به رفتار (behavior-sensitive instruction) دستوری است که تأثیر آن تا حدی بستگی به زمینه‌ای دارد که در آن اجرا می‌شود. برای مثال، پردازنده‌های Intel x86 دستورالعمل‌هایی دارند که ممکن است بسته به اینکه در حالت سیستم اجرا شوند یا حالت کاربر، بر برخی رجیسترها تأثیر بگذارند یا نگذارند. مثالی که در [Nair, ۲۰۰۵b و Smith] آورده شده، دستورالعمل POPF است که ممکن است فلگ فعال‌سازی وقفه را تنظیم کند، اما فقط زمانی که در حالت سیستم اجرا شود.

اکنون به نتیجه مهم زیر می‌رسیم:

برای هر کامپیوتر معمولی، می‌توان یک مانیتور ماشین مجازی ساخت، اگر مجموعه دستورالعمل‌های حساس آن کامپیوتر، زیرمجموعه‌ای از دستورالعمل‌های ویژه باشد.

این جمله یعنی تا زمانی که دستورالعمل‌های حساس هنگام اجرای آن‌ها در حالت کاربر شناسایی شوند، می‌توانیم با اطمینان همه دستورالعمل‌های غیرحساس را به‌صورت بومی روی سخت‌افزار زیربنایی اجرا کنیم. این همچنین به این معناست که هنگام طراحی مجموعه دستورالعمل‌ها، اگر مراقب باشیم که این شرط رعایت شود، مانعی برای مجازی‌سازی بهینه و کارآمد آن مجموعه دستورالعمل ایجاد نمی‌کنیم.

متأسفانه، همه مجموعه دستورالعمل‌ها شامل دستورالعمل‌های حساس ویژه (privileged-only sensitive instructions) نیستند، از جمله شاید محبوب‌ترین آن‌ها، یعنی مجموعه دستورالعمل‌های Intel x86 مشخص شده است که این مجموعه شامل ۱۷ دستورالعمل حساس است که ویژه نیستند [Robin و Irvine، ۲۰۰۰]. به عبارت دیگر، هر یک از این دستورالعمل‌ها می‌توانند در حالت کاربر اجرا شوند بدون اینکه باعث ایجاد تله (trap) به سیستم‌عامل شوند، اما بر نحوه مدیریت منابع توسط سیستم‌عامل تأثیر می‌گذارند. در این موارد، اساساً دو راه‌حل وجود دارد.

راه‌حل اول این است که همه دستورالعمل‌ها را شبیه‌سازی کنیم. البته این کار تأثیر منفی شدیدی روی عملکرد خواهد داشت. برای حل این مشکل، روشی که در VMware پیاده‌سازی شده است [Sugerman و همکاران، ۲۰۰۱]، اسکن کردن فایل اجرایی و قرار دادن کد در اطراف دستورالعمل‌های حساس غیر ویژه برای تغییر مسیر کنترل

برای انتقال کنترل به مانیتور ماشین مجازی. در آنجا، شبیه‌سازی مناسب انجام می‌شود، مثلاً با در نظر گرفتن زمینه‌ای که دستورالعمل قرار بود در آن اجرا شود. نتیجه این است که مجازی‌سازی کامل امکان‌پذیر می‌شود، به این معنی که اجرا می‌تواند بدون تغییر سیستم‌عامل مهمان و یا خود برنامه انجام شود.

راه‌حل جایگزین، استفاده از پارامجازی‌سازی (paravirtualization) است که نیازمند تغییر در سیستم‌عامل مهمان می‌باشد. به طور خاص، سیستم‌عامل مهمان طوری تغییر داده می‌شود که تمام اثرات جانبی اجرای دستورالعمل‌های حساس غیر ویژه در حالت کاربر، که معمولاً در حالت سیستم اجرا می‌شوند، مدیریت شوند. برای مثال، کد می‌تواند بازنویسی شود به گونه‌ای که این دستورالعمل‌ها دیگر اجرا نشوند، یا اگر اجرا شوند، معنا و عملکرد آن‌ها در هر دو حالت کاربر و سیستم یکسان باشد. پارامجازی‌سازی توسط Xen پذیرفته شده است [Barham و همکاران، ۲۰۰۳؛ Chisnall، ۲۰۰۷].

۳.۲.۲ کانتینرها

ماشین‌های مجازی راهی برای اجرای برنامه‌هایی فراهم می‌کنند که به یک محیط عملیاتی خاص، شامل مجموعه دستورالعمل‌ها و سیستم‌عامل وابسته‌اند و می‌خواهند به صورت مستقل روی پلتفرم‌های مختلف اجرا شوند. همان‌طور که دیدیم، این کار ممکن است نیازمند تلاش قابل‌توجهی برای تضمین قابلیت جابجایی و عملکرد باشد. با این حال، اغلب مشاهده می‌کنیم که برنامه‌ها نسبتاً در استفاده از مجموعه دستورالعمل‌ها و سیستم‌عامل پایدار هستند، اما به کتابخانه‌ها و نرم‌افزارهای پشتیبانی خاصی وابسته‌اند. در این موارد، آنچه واقعاً می‌خواهیم این است که برنامه‌های مختلف بتوانند کنار هم اجرا شوند، در حالی که هر کدام از محیط نرم‌افزاری پشتیبانی خاص خود استفاده می‌کنند، بدون اینکه حتی متوجه وجود برنامه‌های دیگر با محیط‌های متفاوت شوند. اینجاست که کانتینرها وارد میدان می‌شوند.

یک کانتینر را می‌توان مجموعه‌ای از فایل‌های باینری (که به آن‌ها ایمیج هم گفته می‌شود) دانست که به‌صورت مشترک محیط نرم‌افزاری لازم برای اجرای برنامه‌ها را تشکیل می‌دهند. ساده‌ترین راه برای تصور کانتینر این است که کاربر هنگام ورود به یک سیستم یونیکس، چه می‌بیند: چندین دایرکتوری استاندارد که شامل برنامه‌های اجرایی، کتابخانه‌ها، مستندات و غیره است. یک پیاده‌سازی ساده و ابتدایی از کانتینر، کپی کردن یک محیط کامل برای یک کاربرد خاص و نصب آن به عنوان یک زیرشاخه از سیستم فایل ریشه (مثلاً `root`) است. با استفاده از دستوری مانند `chroot`، کاربر به آن زیرشاخه هدایت می‌شود و برنامه‌های مختلف را اجرا می‌کند، به طوری که این زیرشاخه اکنون نقش ریشه (`root`) را دارد. یک برنامه دقیقاً کتابخانه‌ها و وابستگی‌های مورد نیاز خود را می‌بیند، در حالی که برنامه‌های داخل کانتینرهای دیگر دیدگاه مخصوص به خود را نسبت به سیستم‌عامل دارند. به این معنا، کانتینر عملاً محیط نرم‌افزاری برنامه را مجازی‌سازی می‌کند.

با این حال، این پیاده‌سازی ساده و ابتدایی از منظر مجازی‌سازی قطعاً کافی نیست. اول اینکه، برنامه‌ها و فرآیندهایی که در کانتینرهای مختلف اجرا می‌شوند

کانتینرها باید از یکدیگر ایزوله باشند. همچنین، کپی کردن کل محیط به صورت کامل کارآمد نیست، مخصوصاً چون انتظار می‌رود بسیاری از کتابخانه‌ها و موارد مشابه بین کانتینرهای مختلف مشترک باشند. در نهایت، مهم است که سیستم‌عاملی که میزبان کانتینرها است، کنترل مناسبی بر استفاده از منابع خود داشته باشد. تمام این موارد در محیط‌های یونیکس (به ویژه لینوکس) از طریق سه مکانیزم مهم مدیریت می‌شوند:

- **Namespace**‌ها: که به مجموعه‌ای از فرآیندهای مرتبط با یک کانتینر دیدگاه جداگانه‌ای از شناسه‌ها و موارد مشابه می‌دهد، مستقل از دیگر کانتینرها.
- فایل سیستم یونیون (`Union file system`): که اجازه می‌دهد چندین فایل سیستم به صورت لایه‌ای ترکیب شوند، به طوری که تنها بالاترین لایه امکان نوشتن دارد (و این لایه بخشی از کانتینر است).
- گروه‌های کنترل (`Control groups` یا `cgroups`): که امکان اعمال محدودیت‌های منابع بر مجموعه‌ای از فرآیندها را فراهم می‌کند.

بیا باید کمی عمیق‌تر به هر یک از این مکانیزم‌ها نگاه کنیم. **Namespace**‌ها برای این لازم هستند که به فرآیندی که داخل یک کانتینر اجرا می‌شود، این توهم را بدهند که در محیطی مستقل و جدا قرار دارد. از این رو، `namespace`‌ها اهمیت زیادی در ایزوله‌سازی کانتینرها از یکدیگر دارند. شاید بهترین مثال، تنظیم `namespace` مربوط به `PID` باشد. چون هر ماشین فقط یک فرآیند `init` دارد (با `PID` برابر ۱)، هر کانتینر باید بتواند فرآیند `"init"` مخصوص به خودش را ببیند. این کار از طریق دستور `unshare` در یونیکس برقرار می‌شود:

```
unshare --pid --fork --mount-proc bash
```

فرآیند فراخواننده را به یک شل جدید می‌برد که در آن دستور `ps -ef` نتایج زیر را نشان می‌دهد:

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	06:27	pts/0	00:00:00	bash
root	2	1	0	06:27	pts/0	00:00:00	ps -ef

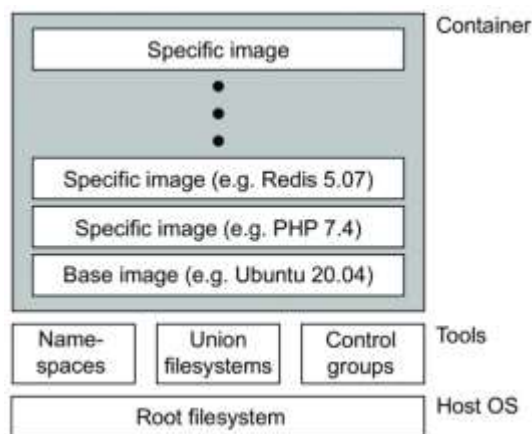
در واقع، مشاهده می‌کنیم که اکنون به نظر می‌رسد مجموعه‌ای جدید از فرآیندها وجود دارد که فقط یکی از آن‌ها PID برابر با ۱ دارد. تمام فرآیندهای دیگر، از دید این شل جدید، نامرئی شده‌اند.

یکی دیگر از مکانیزم‌های مهم، اشتراک‌گذاری کارآمد فایل سیستم‌های موجود است. بسیاری از کانتینرها بر پایه‌ی یک نمونه‌ی مشترک از سیستم‌عامل، مثلاً Ubuntu ۲۰.۴ ساخته می‌شوند. به جای کپی کردن کل آن محیط و نصب به صورت یک زیرشاخه (همان‌طور که قبلاً توضیح داده شد)، می‌توانیم از آن به عنوان لایه پایه استفاده کنیم و بخش‌های دیگر را به صورت لایه‌هایی روی آن قرار دهیم. برای مثال، ممکن است تصمیم بگیریم مجموعه دایرکتوری‌هایی که PHP نسخه ۷.۴ را تشکیل می‌دهند، با نسخه‌ای قدیمی‌تر جایگزین کنیم؛ تنها کافی است دایرکتوری‌های نسخه‌ی قدیمی‌تر را روی دایرکتوری‌های نسخه‌ی ۷.۴ قرار دهیم (stack کنیم). نتیجه این خواهد بود که برنامه‌ی PHP موردنظر از نسخه‌ی قدیمی‌تر استفاده خواهد کرد. توجه داشته باشید که این روش بسیار شبیه به mount کردن یک فایل سیستم روی یک دایرکتوری است.

هر چیزی که در دایرکتوری **dir** قرار داشت، دیگر قابل مشاهده نخواهد بود تا زمانی که فایل سیستم دوباره **unmount** شود. در واقع، ترکیب فایل سیستم‌ها (Union) از طریق فراخوانی‌های متوالی دستور **mount** انجام می‌شود، به طوری که هر لایه به صورت فقط خواندنی (read-only) بارگذاری می‌شود. تنها لایه بالایی قابل نوشتن است و پس از پایان اجرای کانتینر باید به طور صریح ذخیره شود.

در نهایت، برای کنترل اینکه یک کانتینر واقعاً چه میزان از منابع را می‌تواند استفاده کند، سیستم‌های یونیکس **cgroups** (گروه‌های کنترل) را ارائه می‌دهند. در اصل، هنگام ایجاد یک **cgroup**، می‌توان مجموعه‌ای از فرآیندهای در حال اجرا در آن گروه را به لحاظ میزان حافظه اصلی قابل استفاده، اولویت استفاده از پردازنده (CPU) و سایر منابع محدود کرد. به این ترتیب، سیستم‌عامل میزبان می‌تواند از این جلوگیری کند که یک کانتینر، بیش از حد منابع را اشغال کند و مانع اجرای صحیح کانتینرهای دیگر شود.

موارد بسیار دیگری نیز وجود دارند که به ایزوله‌سازی کانتینرها و محدود کردن عملکرد فرآیندها درون کانتینر مربوط می‌شوند. در نهایت، می‌توان یک کانتینر را مانند یک آرشیو از فایل‌ها در نظر گرفت که در جایی از سیستم فایل قرار گرفته‌اند، همراه با مجموعه‌ای خاص از زیردایرکتوری‌های فقط خواندنی مشترک و موجود. فرآیندهایی که در چارچوب یک کانتینر اجرا می‌شوند، با دیدگاهی روبه‌رو هستند که در آن گمان می‌کنند تنها زمینه‌ی موجود، همان کانتینر خودشان است (از طریق استفاده از **namespace**‌ها) و در زمینه استفاده از منابع، توانمندی‌ها و محدودیت‌هایی دارند (از طریق **cgroups** و همچنین محدودیت‌هایی مانند محدودسازی فراخوانی‌های سیستمی). این دیدگاه در شکل ۳.۱۲ خلاصه شده است. علاوه بر مطالب Julia Evans در wizardzines.com، خواننده علاقه‌مند می‌تواند برای مرور کلی فناوری‌های کانتینر به منبع [Pahl و همکاران، ۲۰۱۹] مراجعه کند.

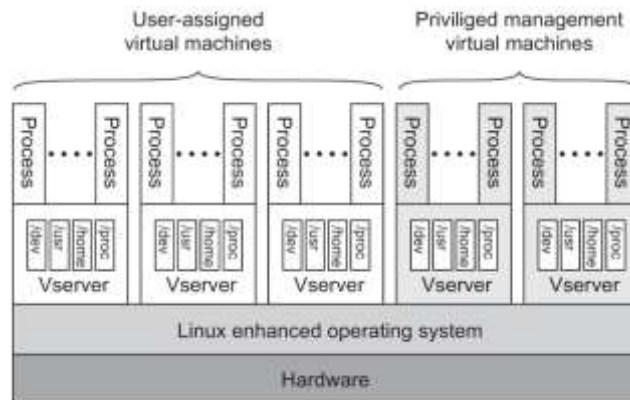


شکل ۳.۱۲: سازمان‌دهی یک کانتینر درون یک محیط میزبان

نمونه‌های زیادی از فناوری کانتینر وجود دارد، اما در اینجا جالب است به یک مورد خاص نگاه کنیم که در آن این فناوری برای توسعه‌ی یک کلاستر گسترده از رایانه‌ها به کار گرفته شد؛ حتی قبل از آن که کانتینرها در زمینه رایانش ابری محبوب شوند.

PlanetLab یک سیستم توزیع‌شده‌ی مشارکتی بود که در آن سازمان‌های مختلف، هرکدام یک یا چند رایانه اهدا می‌کردند که در مجموع به صدها گره (node) می‌رسید. این رایانه‌ها با هم یک کلاستر سرور تک‌لایه‌ای (۱-tier) را تشکیل می‌دادند که در آن دسترسی، پردازش و ذخیره‌سازی می‌توانست به‌طور مستقل روی هر گره انجام شود. مدیریت PlanetLab، بنا به ضرورت، تقریباً به‌طور کامل توزیع‌شده بود. این پروژه در سال ۲۰۲۰ پایان یافت.

سازمان‌دهی کلی در PlanetLab، هر سازمان شرکت‌کننده یک یا چند گره (یعنی رایانه) اهدا می‌کرد که سپس میان تمام کاربران PlanetLab به اشتراک گذاشته می‌شد. هر گره به شکلی که در شکل ۳.۱۳ نشان داده شده سازمان‌دهی شده بود. در این ساختار، دو مؤلفه‌ی مهم وجود دارد [Baviera و همکاران، ۲۰۰۴؛ Peterson و همکاران، ۲۰۰۶]. اولین مؤلفه، مانیتور ماشین مجازی (VMM) است که نسخه‌ای تقویت‌شده از سیستم‌عامل لینوکس می‌باشد؛ در واقع، نسخه‌ای که قابلیت پشتیبانی از کانتینرها را، طبق آنچه در بالا شرح داده شد، دارد. این تقویت‌ها عمدتاً شامل تنظیماتی برای پشتیبانی از مؤلفه دوم هستند، یعنی Vserverهای لینوکسی. یک Vserver در اصل همان کانتینر در حال اجرا است.

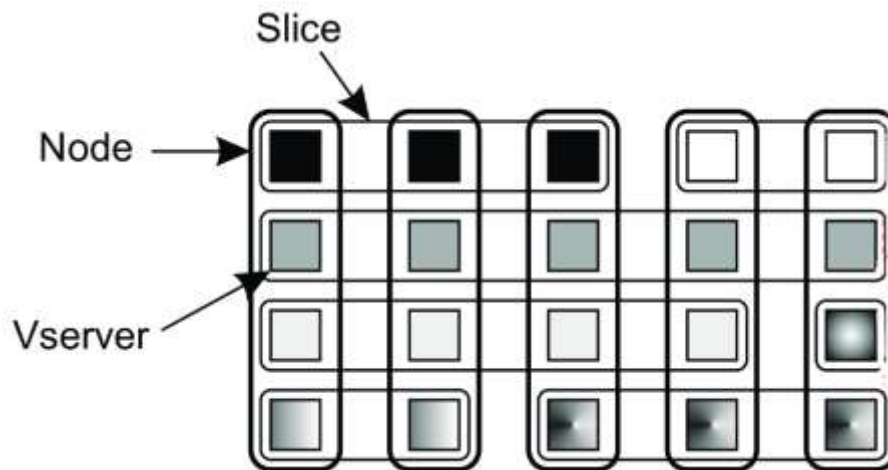


شکل ۳.۱۳: سازمان‌دهی پایه‌ای یک گره (node) در PlanetLab.

مانیتور ماشین مجازی لینوکس (Linux VMM) تضمین می‌کرد که Vserverها از هم جدا باشند: فرآیندهای داخل Vserverهای مختلف به‌صورت هم‌زمان و مستقل اجرا می‌شوند و هر کدام تنها از بسته‌ها و برنامه‌های نرم‌افزاری موجود در محیط خود استفاده می‌کنند. ایزوله‌سازی بین فرآیندهای درون Vserverهای مختلف بسیار دقیق است. برای مثال، دو فرآیند در دو Vserver مختلف ممکن است شناسه کاربری (User ID) یکسانی داشته باشند، اما این به معنی این نیست که آن‌ها متعلق به یک کاربر باشند. این جداسازی به طور قابل توجهی پشتیبانی از کاربران سازمان‌های مختلف را که می‌خواستند از PlanetLab به عنوان بستری برای آزمایش سیستم‌ها و برنامه‌های توزیع‌شده کاملاً متفاوت استفاده کنند، ساده‌تر می‌کرد. توجه کنید که این جداسازی دقیقاً همان چیزی است که از طریق دستور unshare تحقق می‌یابد.

برای پشتیبانی از چنین آزمایش‌هایی، PlanetLab از مفهومی به نام “اسلایس” (slice) استفاده می‌کرد؛ هر اسلایس مجموعه‌ای از Vserverها بود که هر Vserver روی یک گره (node) متفاوت اجرا می‌شد، همان‌طور که در شکل ۳.۱۴ نشان داده شده است. بنابراین، می‌توان یک اسلایس را به‌صورت یک کلاستر سرور مجازی در نظر گرفت که توسط مجموعه‌ای از کانتینرها پیاده‌سازی شده و از طریق یک شبکه گسترده (wide-area network) به هم متصل هستند.

مرکز مدیریت منابع در PlanetLab، «مدیر گره (node manager)» بود. هر گره دارای یک مدیر گره بود که در قالب یک Vserver جداگانه پیاده‌سازی شده بود. وظیفه‌ی اصلی این مدیر، ایجاد سایر Vservers روی گره‌ای بود که کنترل آن را برعهده داشت و نیز مدیریت تخصیص منابع. برای ایجاد یک اسلایس جدید، هر گره یک سرویس به نام سرویس ایجاد اسلایس (SCS) را نیز اجرا می‌کرد. این سرویس می‌توانست با مدیر گره تماس بگیرد تا از او بخواهد یک Vserver بسازد و منابعی را به آن اختصاص دهد. مدیر گره به‌طور مستقیم از طریق شبکه قابل دسترسی نبود، تا تمرکزش فقط روی مدیریت منابع محلی باقی بماند. از طرف دیگر، سرویس SCS نیز فقط از هر کسی درخواست ایجاد اسلایس را نمی‌پذیرفت؛ تنها نهادهای خاصی با عنوان «مرجع اسلایس (slice authority)» اجازه داشتند که درخواست ایجاد اسلایس ثبت کنند. هر مرجع اسلایس به مجموعه‌ای از گره‌ها دسترسی داشت. ساده‌ترین مدل این بود که تنها یک مرجع اسلایس متمرکز وجود داشته باشد که مجاز به ایجاد اسلایس در همه گره‌ها باشد. در عمل، این مرجع همان نهادی بود که به کاربران اجازه می‌داد تا بتوانند فعالیت خود را در PlanetLab آغاز کنند.



شکل ۳.۱۴: اصل مفهوم یک اسلایس در PlanetLab، که مجموعه‌ای از Vservers مرتبط را در گره‌های مختلف نشان می‌دهد.

مدیریت و تخصیص منابع از طریق یک مشخصه ویژه به نام "تعریف منابع" انجام می‌شود. این مشخصه محدوده زمانی‌ای را مشخص می‌کند که در آن، برخی امکانات در دسترس قرار می‌گیرند. این امکانات شامل فضای ذخیره‌سازی، شناسه‌های پرونده، پهنای باند تبادل اطلاعات، نقاط ارتباطی در سامانه انتقال داده، حافظه اصلی و توان پردازشی می‌شود. هر "تعریف منابع" دارای شناسه‌ای یکتا و جهانی است که به آن "گواهی دسترسی به منابع" می‌گویند. با استفاده از این شناسه، مدیر بخش می‌تواند اطلاعات مربوطه را در فهرست داخلی خود جستجو کند.

منابع به واحدهایی به نام "برش‌ها" مرتبط هستند. به عبارت دیگر، برای استفاده از امکانات، باید یک "برش" ایجاد شود. هر "برش" با یک ارائه‌دهنده خدمات مرتبط است که در واقع یک شخصیت حقیقی یا حقوقی دارای حساب کاربری در سامانه موردنظر محسوب می‌شود. هر "برش" سپس با یک مجموعه شامل "شناسه اصلی" و "نشان برش" قابل شناسایی است، که در آن، "شناسه اصلی" ارائه‌دهنده خدمات را مشخص می‌کند و "نشان برش" یک شناسه است که توسط همین ارائه‌دهنده تعیین می‌شود.

منابع به بخش‌هایی از سیستم اختصاص داده می‌شوند. به عبارت دیگر، برای استفاده از منابع، لازم بود که یک بخش ایجاد شود. هر بخش به یک ارائه‌دهنده خدمات مرتبط بود که می‌توان آن را به‌عنوان یک موجودیت دارای حساب کاربری در سیستم PlanetLab در نظر گرفت. هر بخش را می‌توان از طریق ترکیب (شناسه اصلی، برچسب بخش) شناسایی کرد؛ جایی که شناسه اصلی ارائه‌دهنده خدمات را مشخص می‌کند و برچسب بخش یک شناسه‌ای است که توسط ارائه‌دهنده انتخاب می‌شود.

سرورهای مجازی (Vservers)

در این بخش، به بررسی سرورهای مجازی در Planet Lab می‌پردازیم که توسط Soltesz et al. (۲۰۰۷) توضیح و ارزیابی شده‌اند. یک سرور مجازی به صورت یک محفظه (کانتینر) سازماندهی شده است. وظیفه اصلی سرور مجازی فقط پشتیبانی از یک گروه پردازشی و جدا نگه داشتن آن از پردازش‌هایی است که تحت نظارت سرورهای مجازی دیگر اجرا می‌شود.

یکی از مزیت‌های مهم استفاده از کانتینرها در مجازی‌سازی، در مقایسه با اجرای سیستم‌عامل‌های مجازی جداگانه، این است که تخصیص منابع معمولاً ساده‌تر انجام می‌شود. به طور خاص، امکان بیش‌ازحد تخصیص منابع با اجازه دادن به تخصیص پویا وجود داشت، مشابه نحوه تخصیص منابع به فرایندهای عادی.

معمولاً هنگام استفاده از یک سیستم‌عامل میهمان، باید تعداد مشخصی از منابع (به‌ویژه حافظه اصلی) از پیش به آن اختصاص داده شود. با توجه به اینکه نودهایی که توسط سازمان‌های مشارکت‌کننده در PlanetLab ارائه می‌شوند تنها چند گیگابایت حافظه اصلی دارند، قابل تصور است که حافظه به یک منبع کمیاب تبدیل شود.

بنابراین، تخصیص پویا حافظه ضروری بود تا امکان اجرای هم‌زمان ده‌ها کانتینر روی یک نود واحد فراهم شود. Vserver ها برای این نوع مدیریت منابع ایده‌آل بودند، در حالی که سیستم‌عامل‌های کامل پشتیبانی از چنین مواردی را دشوارتر می‌کنند.

البته، این امر نمی‌توانست مانع مصرف بیش از حد حافظه توسط یک Vserver در یک نود شلوغ شود. سیاست PlanetLab در این شرایط ساده بود: اگر Vserver حافظه زیادی مصرف می‌کرد و فضای swap تقریباً پر شده بود، آن سرور بازنشانی می‌شد.

۳.۲.۳ مقایسه ماشین‌های مجازی و کانتینرها

از زمانی که کانتینرها محبوب شدند، که عمدتاً به دلیل معرفی آن‌ها توسط Docker بود، بحث‌های داغی در مورد اینکه کدام یک بهتر است: ماشین‌های مجازی یا کانتینرها شکل گرفته است.

این بحث اغلب تحت تأثیر استفاده نادرست از اصطلاحات قرار گرفته است، مانند مقایسه "کانتینرهای سبک" با "ماشین‌های مجازی سنگین"، که بلافاصله به بیان‌های عملکردی غیرمستند منجر می‌شود (به‌گونه‌ای که سنگین بودن به معنی کند بودن تلقی شود). اما واقعیت پیچیده‌تر از این است، و درک فناوری ماشین‌های مجازی و کانتینرها می‌تواند در تصمیم‌گیری‌های بهتر برای انتخاب فناوری مناسب کمک کند. در این بخش، ما به بررسی یک جنبه خاص و مهم می‌پردازیم: عملکرد. موضوع پرتابل بودن (قابلیت جابه‌جایی) را بعداً در بحث مهاجرت کد بررسی خواهیم کرد مطالعه‌ای سیستماتیک توسط Sharma et al. (۲۰۱۶) در مورد مقایسه کانتینرهای لینوکس (LXC) با ماشین‌های مجازی لینوکس (KVM) انجام شد. اگر صرفاً مقایسه اولیه‌ای را در نظر بگیریم که در آن یک برنامه یا در داخل یک کانتینر اجرا شود یا روی یک ماشین مجازی، تفاوت‌هایی به نفع کانتینرها مشاهده می‌شود، اما این تفاوت‌ها چندان زیاد نیستند، مگر در مورد عملیات I/O. در این حالت، ماشین‌های مجازی عملکرد ضعیف‌تری نسبت به کانتینرها دارند. این موضوع نباید تعجب‌آور باشد، زیرا در I/O است، سیستم‌عامل نقش حیاتی ایفا می‌کند و باید دستورات سطح بالای زیادی را اجرا کند. با این حال، حتی بدیهیات نیز ممکن است شگفتی‌هایی به همراه داشته باشند.

در مطالعه‌ای جدیدتر توسط van Rijn و Rellermeyer (۲۰۲۱) در مقایسه فناوری‌های مختلف کانتینری با روش‌های مبتنی بر ماشین مجازی نشان داده شده است که تفاوت‌ها اغلب ناچیز هستند، حتی در مورد عملیات دیسک و شبکه. یکی از دلایل این تفاوت کم، کش کردن نتایج توسط سیستم‌عامل میزبان در حافظه اصلی است. به عبارت دیگر، هنگام اجرای عملیات I/O واقعی، بسیاری از عملیات بعدی روی داده‌های موجود در حافظه انجام می‌شوند، نه داده‌هایی که روی دیسک ذخیره شده‌اند. این موضوع ارزیابی عملکرد را دشوارتر می‌کند اما رفتار واقعی

برنامه‌ها را نیز بازتاب می‌دهد. علاوه بر این، در هنگام اجرای ارزیابی‌های سطح برنامه، مانند ارزیابی‌هایی که برای MySQL انجام می‌شود، ممکن است تفاوت‌هایی وجود داشته باشد، اما این تفاوت‌ها معمولاً کوچک هستند.

با این حال، در مجموع، ماشین‌های مجازی بار پردازشی بیشتری نسبت به کانتینرها دارند. از طرفی، در سناریوهای واقعی، معمولاً چندین برنامه به طور هم‌زمان اجرا می‌شوند. سوال کلیدی این است که تا چه حد عملکرد یک برنامه روی دیگری تأثیر می‌گذارد؟ این موضوع اساساً به چگونگی زمان‌بندی منابع بین رقبا مربوط می‌شود. به‌طور کلی، کانتینرها در جداسازی برنامه‌های مستقل با چالش بیشتری مواجه‌اند و زمان‌بندی برای مصرف CPU و عملکرد دیسک از طریق ماشین‌های مجازی بهتر مدیریت می‌شود. به‌طور طبیعی، اجرای هم‌زمان چندین برنامه امری رایج است. سوال اصلی این است که تا چه حد عملکرد یک برنامه بر دیگری تأثیر می‌گذارد؟ در واقع، این موضوع به نحوه زمان‌بندی و تخصیص منابع میان برنامه‌های رقابتی برمی‌گردد. به‌طور کلی، کانتینرها در جداسازی برنامه‌های مستقل با چالش بیشتری مواجه‌اند و زمان‌بندی مصرف CPU و عملکرد دیسک در ماشین‌های مجازی بهتر مدیریت می‌شود. مطالعات اخیر، از جمله van Rijn و Rellermeier (۲۰۲۱) نشان داده‌اند که با گذر زمان، بهبودهای قابل توجهی انجام شده است و نیازی نیست که فناوری‌های مجازی‌سازی عملکرد بسیار کندتری نسبت به اجرای مستقیم برنامه‌ها روی سیستم عامل میزبان داشته باشند.

۳.۲۴ کاربرد ماشین‌های مجازی در سیستم‌های توزیع‌شده

از دیدگاه سیستم‌های توزیع‌شده، مهم‌ترین کاربرد مجازی‌سازی در رایانش ابری دیده می‌شود. همان‌طور که در بخش ۱.۳.۱ اشاره کردیم، ارائه‌دهندگان خدمات ابری به‌طور کلی سه نوع سرویس اصلی ارائه می‌دهند:

- شامل زیرساخت‌های اساسی → Infrastructure-as-a-Service (IaaS)
- ارائه خدمات در سطح پلتفرم → Platform-as-a-Service (PaaS)
- شامل برنامه‌های کاربردی واقعی → Software-as-a-Service (SaaS)

مجازی‌سازی نقش کلیدی در IaaS ایفا می‌کند. به جای اجاره یک ماشین فیزیکی، ارائه‌دهنده خدمات ابری یک ماشین مجازی (مانیتور) را در اختیار مشتری قرار می‌دهد که ممکن است منابع یک ماشین فیزیکی را با سایر کاربران به اشتراک بگذارد یا نگذارد. مزیت مهم مجازی‌سازی این است که امکان تفکیک تقریباً کامل میان مشتریان را فراهم می‌کند، به‌گونه‌ای که کاربران تصور می‌کنند یک ماشین فیزیکی اختصاصی را اجاره کرده‌اند. با این حال، تفکیک منابع هرگز کاملاً کامل نیست، زیرا منابع فیزیکی واقعی همچنان بین کاربران به اشتراک گذاشته می‌شوند، که در نهایت می‌تواند منجر به کاهش عملکرد قابل مشاهده شود.

Amazon EC2 به کاربران امکان ایجاد محیطی متشکل از چندین سرور مجازی متصل به شبکه را می‌دهد که اساس یک سیستم توزیع‌شده را شکل می‌دهند.

برای سهولت استفاده، تعداد زیادی تصاویر از پیش پیکربندی‌شده تحت عنوان Amazon Machine Images (AMIs) در دسترس هستند.

AMI یک بسته نرم‌افزاری قابل نصب است که شامل کرنل سیستم عامل به همراه چندین سرویس می‌شود. یک تصویر LAMP شامل: کرنل لینوکس، سرور وب Apache، سیستم پایگاه داده MySQL، کتابخانه‌های PHP، علاوه بر این، تصاویر پیشرفته‌تر نیز موجود هستند که نرم‌افزارهای اضافی را شامل می‌شوند. برخی از تصاویر AMI مبتنی بر کرنل‌های دیگر Unix یا ویندوز هستند. بدین معنا، یک AMI اساساً مشابه یک دیسک بوت است، اگرچه تفاوت‌هایی وجود دارد که بعداً به آن‌ها می‌پردازیم.

مشتری EC2 باید یک AMI را انتخاب و پیکربندی کند. سپس می‌تواند AMI را اجرا کند که نتیجه آن یک EC2 Instance است؛ یعنی ماشین مجازی واقعی که میزبان برنامه‌های کاربردی مشتری خواهد بود. مکان اجرای یک EC2 Instance معمولاً برای مشتری مشخص نیست. این ماشین مجازی بر روی یک سرور فیزیکی اجرا می‌شود، اما مکان دقیق این سرور پنهان باقی می‌ماند.

نزدیک‌ترین راه برای مشخص کردن موقعیت اجرای یک Instance، انتخاب یکی از مناطق جغرافیایی ارائه‌شده توسط Amazon است: ایالات متحده، آمریکای جنوبی، اروپا، آسیا

هر EC2 Instance دو آدرس IP دریافت می‌کند: آدرس خصوصی: برای ارتباط داخلی بین چندین Instance. با استفاده از شبکه داخلی EC2 آدرس عمومی: برای ارتباط با کاربران اینترنت که می‌توانند به Instance دسترسی داشته باشند آدرس عمومی به آدرس خصوصی با استفاده از فناوری NAT (ترجمه آدرس شبکه) نگاشت می‌شود. یکی از روش‌های مدیریت آسان یک EC2 Instance استفاده از اتصال SSH است Amazon. ابزارهای لازم برای تولید کلیدهای مناسب جهت این ارتباط را فراهم می‌کند.

محیط EC2 امکانات مختلفی را برای تنظیم منابع Instance در اختیار قرار می‌دهد:

- CPU → (GPU از جمله) امکان انتخاب تعداد و نوع هسته پردازشی
- Memory → Instance میزان حافظه اصلی اختصاص‌یافته به
- Storage → میزان فضای ذخیره‌سازی محلی
- Platform → ۳۲ بیتی یا ۶۴ بیتی انتخاب معماری پردازشی
- Networking → تعیین ظرفیت پهنای باند قابل استفاده

علاوه بر منابع اولیه، کاربران می‌توانند منابع اضافی مانند رابط شبکه‌ای اضافه را نیز درخواست کنند. حافظه محلی که به یک Instance اختصاص داده می‌شود موقت است؛ به این معنا که با توقف Instance، تمام داده‌های ذخیره‌شده محلی از بین می‌رود. برای جلوگیری از حذف اطلاعات، مشتری باید داده‌ها را به یک فضای ذخیره‌سازی دائمی منتقل کند. راهکار اول: استفاده از سرویس ذخیره‌سازی ساده آمازون (Amazon S3) برای ذخیره داده‌های پایدار راهکار دوم: اتصال یک دستگاه ذخیره‌سازی که با Elastic Block Store (EBS) آمازون نگاشت شده باشد EBS یک سرویس ذخیره‌سازی بلوکی است که همانند یک دیسک سخت مجازی عمل می‌کند. داده‌های ذخیره‌شده در EBS پس از توقف Instance باقی می‌مانند و می‌توان دوباره یک دستگاه EBS را به هر Instance دیگری متصل کرد.

بدون ورود به جزئیات پیچیده، IaaS در EC2 به مشتریان امکان می‌دهد تعداد زیادی ماشین مجازی ایجاد کنند که هر یک با منابع مورد نیاز پیکربندی شده است و از طریق شبکه IP با یکدیگر ارتباط برقرار می‌کنند. علاوه بر این، این ماشین‌های مجازی از هر نقطه‌ای اینترنت قابل دسترسی هستند (البته فقط در صورتی که کاربر اعتبارنامه‌های مناسب داشته باشد)

به همین دلیل، EC2، مانند سایر ارائه‌دهندگان IaaS، امکان راه‌اندازی یک سیستم توزیع‌شده کامل را فراهم می‌کند که شامل سرورهای مجازی شبکه‌ای و اجرای برنامه‌های توزیع‌شده کاربران است. همچنین مشتریان نیازی به مدیریت ماشین‌های فیزیکی ندارند، که خود یک مزیت بزرگ در دنیای رایانش ابری مدرن محسوب می‌شود

۳.۳ مشتری‌ها (Clients)

در فصل‌های قبلی، مدل کلاینت-سرور، نقش‌های کلاینت‌ها و سرورها و نحوه تعامل آن‌ها را بررسی کردیم. اکنون، نگاهی دقیق‌تر به ساختار کلاینت‌ها و سرورها خواهیم داشت. در این بخش، ابتدا کلاینت‌ها را بررسی می‌کنیم و سرورها در بخش بعدی مورد بحث قرار می‌گیرند.

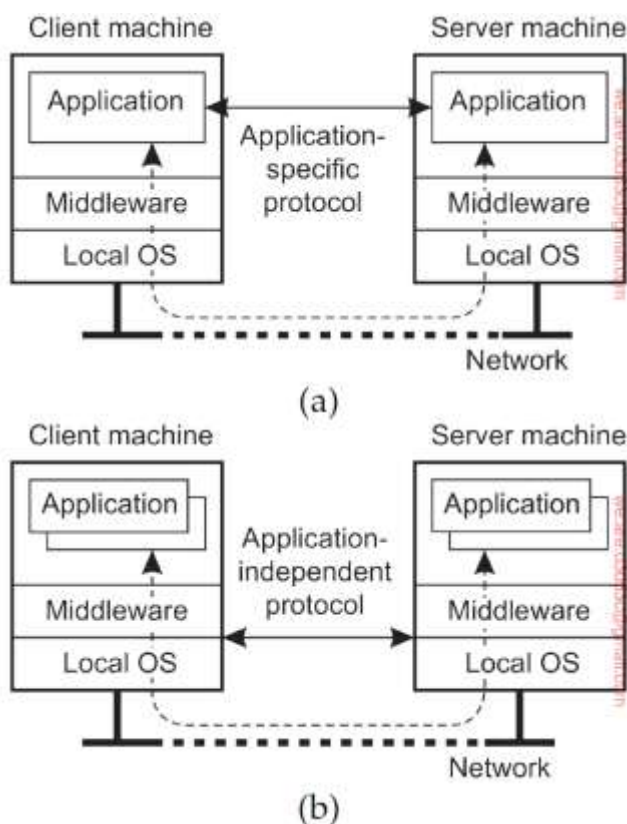
۳.۳.۱ رابط‌های کاربری شبکه‌ای

یکی از وظایف اصلی ماشین‌های کلاینت فراهم کردن ابزارهایی برای تعامل کاربران با سرورهای راه دور است. به طور کلی، این ارتباط را می‌توان به دو روش برقرار کرد. ابتدا ارتباط مستقیم بین کلاینت و سرویس راه دور در این روش، برای هر سرویس راه دور، یک برنامه کلاینت متناظر روی ماشین کاربر اجرا می‌شود که می‌تواند از طریق شبکه با سرویس ارتباط برقرار کند. نمونه‌ای از این مدل: یک برنامه تقویم روی گوشی هوشمند کاربر که باید با یک تقویم راه دور (احتمالاً مشترک) همگام‌سازی شود. در این حالت، یک پروتکل سطح برنامه وظیفه‌ی همگام‌سازی را برعهده دارد مانند شکل ۳.۱۵(a).

دومین راهکار دسترسی مستقیم به سرویس‌های راه دور در این روش، کلاینت تنها یک رابط کاربری ساده برای دسترسی به سرویس راه دور فراهم می‌کند. در واقع، ماشین کلاینت فقط به عنوان یک ترمینال عمل می‌کند و نیازی به ذخیره‌سازی محلی ندارد، که منجر به یک راه‌حل مستقل از نوع برنامه می‌شود همان‌طور که در شکل ۳.۱۵(b) نشان داده شده است.

در این مدل، تمام پردازش‌ها و ذخیره‌سازی داده‌ها در سرور انجام می‌شود.

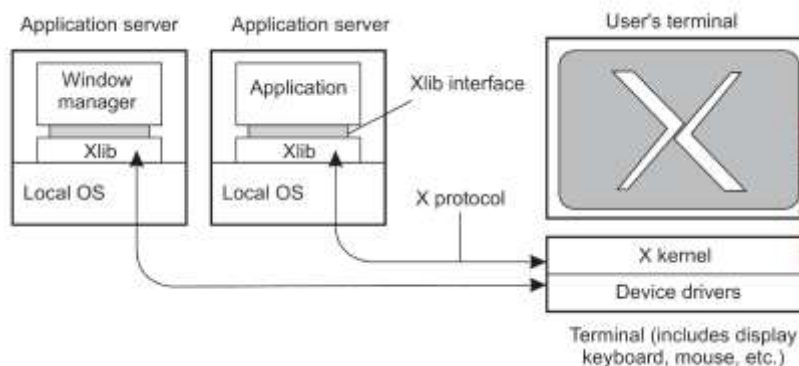
این رویکرد کلاینت سبک (Thin Client) با افزایش اتصال اینترنت و رشد استفاده از دستگاه‌های همراه توجه زیادی را به خود جلب کرده است. راه‌حل‌های Thin Client محبوبیت زیادی دارند، زیرا مدیریت سیستم را ساده‌تر می‌کنند.



شکل ۳.۱۵ (a): یک برنامه شبکه‌ای با پروتکل اختصاصی خود (b) یک راهکار عمومی برای دسترسی به برنامه‌های راه دور

مثال: سیستم پنجره‌ای X (X Window System)

سیستم پنجره‌ای X یکی از قدیمی‌ترین و همچنان پرکاربردترین رابط‌های کاربری شبکه‌ای است. این سیستم، که معمولاً به صورت X شناخته می‌شود، برای کنترل پایانه‌های گرافیکی مبتنی بر بیت به کار می‌رود. این پایانه‌ها شامل نمایشگر، صفحه کلید، و دستگاه اشاره گر مانند ماوس هستند. علاوه بر پشتیبانی از پایانه‌های سنتی در رایانه‌های رومیزی و ایستگاه‌های کاری، X از دستگاه‌های مدرن مانند صفحه‌نمایش‌های لمسی در تبلت‌ها و تلفن‌های هوشمند نیز پشتیبانی می‌کند. به نوعی می‌توان X را بخشی از سیستم‌عامل دانست که وظیفه کنترل پایانه‌ها را بر عهده دارد. هسته سیستم X هسته اصلی این سیستم را چیزی تشکیل می‌دهد که می‌توان آن را "X Kernel" نامید. این هسته شامل تمام درایورهای مخصوص پایانه‌ها است و وابستگی زیادی به سخت‌افزار دارد. هسته X یک رابط سطح پایین برای کنترل صفحه‌نمایش ارائه می‌دهد و همچنین وظیفه دریافت رویدادهای ورودی از صفحه کلید و ماوس را بر عهده دارد. این رابط از طریق کتابخانه‌ای به نام Xlib در اختیار برنامه‌ها قرار می‌گیرد. ساختار Xlib در شکل ۳.۱۶ نشان داده شده است. با این حال، Xlib به ندرت مستقیماً توسط برنامه‌ها استفاده می‌شود. در عوض، برنامه‌ها از مجموعه ابزارهایی استفاده می‌کنند که روی Xlib پیاده‌سازی شده‌اند.



شکل ۳.۱۶: ساختار پایه‌ای سیستم پنجره‌ای X

یکی از ویژگی‌های سیستم X این است که هسته X و برنامه‌های X لزوماً روی یک ماشین واحد اجرا نمی‌شوند. X یک پروتکل سطح برنامه ارائه می‌دهد که به Xlib اجازه تبادل داده و رویدادها با هسته X را می‌دهد. به عنوان مثال، Xlib می‌تواند درخواست‌هایی به هسته X ارسال کند تا یک پنجره ایجاد یا حذف کند، رنگ‌ها را تنظیم کند، نوع نشانگر (cursor) را مشخص کند، و بسیاری از تنظیمات دیگر را اعمال کند. در مقابل، هسته X به رویدادهای محلی پاسخ می‌دهد، مانند ورودی‌های صفحه کلید و ماوس، و بسته‌های رویداد را به Xlib ارسال می‌کند. چندین برنامه می‌توانند هم‌زمان با هسته X ارتباط برقرار کنند. با این حال، یک برنامه خاص وجود دارد که حقوق ویژه‌ای دارد و به عنوان مدیر پنجره (Window Manager) شناخته می‌شود. مدیر پنجره وظیفه دارد ظاهر و نحوه تعامل نمایشگر با کاربر را تعیین کند. مثلاً می‌تواند مشخص کند که پنجره‌ها چگونه تزیین شوند، دکمه‌های اضافی را تعریف کند، نحوه قرارگیری پنجره‌ها روی نمایشگر را تعیین کند، در عمل، بخش زیادی از تعامل بین برنامه‌ها و پایانه X از طریق مدیر پنجره انجام می‌شود.

سیستم پنجره‌ای X به گونه‌ای طراحی شده که هسته X درخواست‌های مربوط به مدیریت نمایشگر را دریافت می‌کند. این درخواست‌ها از سوی برنامه‌ها (ممکن است راه دور باشند) ارسال می‌شوند، بنابراین، هسته X نقش یک سرور را ایفا می‌کند، در حالی که برنامه‌ها به عنوان کلاینت عمل می‌کنند. این اصطلاحات توسط سیستم X پذیرفته شده‌اند، اما گاهی ممکن است باعث سردرگمی شوند.

محاسبات شبکه‌ای مبتنی بر کلاینت سبک (Thin Client)

برنامه‌ها نمایشگر را با استفاده از دستورات خاص X کنترل می‌کنند. این دستورات از طریق شبکه ارسال شده و سپس توسط هسته X اجرا می‌شوند. به‌طور ایده‌آل، برنامه‌های نوشته‌شده برای X باید منطق برنامه را از دستورات رابط کاربری جدا کنند، اما اغلب این‌گونه نیست. طبق گزارش Lai و Nieh (۲۰۰۲)، بخش زیادی از منطق برنامه و تعامل کاربر به‌شدت به یکدیگر وابسته هستند. این یعنی برنامه‌ها درخواست‌های زیادی به هسته X ارسال می‌کنند و منتظر پاسخ می‌مانند تا بتوانند گام بعدی را انجام دهند. این رفتار همگام (synchronous) ممکن است باعث کاهش عملکرد در شبکه‌های گسترده با زمان تأخیر بالا شود. چندین روش برای حل این مشکل ارائه شده است. یکی از این روش‌ها مهندسی مجدد پروتکل X است، همان‌طور که در (Pinzari, ۲۰۰۳) انجام شده است. یکی از اهداف اصلی این پروژه کاهش مصرف پهنای باند از طریق کوچک‌سازی اندازه پیام‌های X است. پیام‌ها شامل دو بخش هستند، بخش ثابت، به‌عنوان شناسه پیام، بخش متغیر، داده‌های متغیر پیام چندین پیام ممکن است شناسه یکسان داشته باشند، که معمولاً داده‌های مشابهی را شامل می‌شوند. این خاصیت برای ارسال فقط تفاوت‌های بین پیام‌های مشابه استفاده می‌شود. با نگهداشتن شناسه‌ها در فرستنده و گیرنده، دریافت‌کننده می‌تواند پیام‌ها را رمزگشایی کند. به لطف این فناوری، کاهش مصرف پهنای باند تا ۱۰۰۰ برابر گزارش شده است، که امکان اجرای X روی لینک‌های کم‌سرعت با پهنای باند ۹۶۰۰ kbps را فراهم می‌کند.

محققان و متخصصان راهکاری جایگزین برای سیستم پنجره‌ای X پیشنهاد کرده‌اند که در آن، برنامه کنترل کامل نمایشگر از راه دور را به دست می‌گیرد، یعنی تا سطح پیکسل. در این روش، تغییرات در بیت‌مپ نمایشگر از طریق شبکه ارسال می‌شود و سپس مستقیماً به بافر فریم محلی انتقال می‌یابد.

یک نمونه شناخته‌شده از این رویکرد، Virtual Network Computing (VNC) است که توسط Richardson et al. (۱۹۹۸) معرفی شد و از اواخر دهه ۱۹۹۰ تاکنون مورد استفاده قرار گرفته است. کنترل نمایشگر توسط برنامه‌ها نیازمند روش‌های رمزگذاری پیچیده است تا از مشکلات پهنای باند جلوگیری شود. مثلاً نمایش یک ویدیو با نرخ ۳۰ فریم در ثانیه روی صفحه‌نمایش ۳۲۰×۲۴۰ پیکسلی، با رمزگذاری هر پیکسل در ۲۴ بیت، در صورت عدم استفاده از روش فشرده‌سازی کارآمد به حدود ۵۳ مگابیت بر ثانیه پهنای باند نیاز خواهد داشت. در عمل، تکنیک‌های رمزگذاری مختلفی استفاده می‌شود، اما انتخاب بهترین روش معمولاً به نوع برنامه بستگی دارد.

۳.۳.۲ محیط دسکتاپ مجازی

با رشد رایانش ابری و افزایش تعداد برنامه‌های ابری، این امکان فراهم شد که ابر به یک محیط دسکتاپ مجازی برای کاربران نهایی تبدیل شود. تنها چیزی که لازم بود، نرم‌افزار سمت کلاینت برای دسترسی به آن محیط دسکتاپ بود.

یکی از اولین شرکت‌هایی که این مدل را ارائه کرد، گوگل بود که Chrome OS را معرفی کرد. ایده‌ی اصلی بسیار ساده است، مرورگر نقش رابط کاربری دسکتاپ محلی را ایفا کند. در کنار مرورگر، چندین برنامه مستقل نیز در دسترس باشند که در نهایت با یک نسخه ابری متناظر هماهنگ شوند. در این مدل دسکتاپ ابری، تجربه‌ی کاربری مشابه آنچه در گوشی‌های هوشمند مدرن ارائه می‌شود شکل می‌گیرد. با افزایش روند تبدیل برنامه‌ها به افزونه‌های مرورگر، می‌توان مشاهده کرد که مرورگر به‌تدریج نقش رابط کاربری سیستم‌عامل را به عهده می‌گیرد.

ساختار داخلی یک مرورگر وب

برای درک بهتر فرآیندهای درون یک مرورگر، نگاهی دقیق‌تر به ساختار مرورگر Chrome خواهیم داشت، که گفته می‌شود پرمصرف‌ترین مرورگر در میان تمامی مرورگرها است. مرورگر Chrome یک نرم‌افزار پیچیده است که بیش از ۲۵ میلیون خط کد دارد، که از نظر اندازه قابل‌مقایسه با کرنل لینوکس است. در حالی که وب در ابتدای کار فقط شامل صفحات ساده HTML بود، امروزه علاوه بر زبان نشانه‌گذاری پیشرفته، مجموعه‌ای از ابزارهای تعاملی و زبان‌های برنامه‌نویسی سمت کلاینت نیز به کار گرفته می‌شوند. ۳.۱۷.

در مرکز سیستم مرورگر، بارگذار منابع (Resource Loader) قرار دارد که وظیفه دارد محتوا را از یک سرور وب دریافت کند. ابتدا یک فایل HTML بارگیری می‌شود، سپس به‌صورت موازی، اتصال‌هایی برای دریافت سایر منابع مرتبط با صفحه برقرار می‌شود، این فرایند معمولاً از طریق رشته‌های پردازشی جداگانه انجام می‌شود. نکته مهم بارگذار منابع نیاز دارد که بخشی از فایل HTML اصلی را تجزیه کند تا مشخص شود کدام منابع دیگر باید بارگیری شوند. بخش اصلی پردازش HTML اما توسط یک مولفه جداگانه انجام می‌شود، که وظیفه دارد مدل شیء سند (DOM) را بسازد. مدل شیء سند (Document Object Model – DOM) یک نمایش درختی از فایل HTML است، که ساختار صفحه را تعریف می‌کند. در حالی که DOM ساختار صفحه را نشان می‌دهد، اطلاعات مربوط به سبک‌دهی در سندی جداگانه تعریف می‌شود. مثلاً این سند مشخص می‌کند که هر پاراگراف باید با یک رنگ خاص و یک فونت مشخص نمایش داده شود. اطلاعات سبک‌دهی به‌صورت جداگانه تجزیه و به DOM اضافه می‌شود، که در نهایت منجر به تشکیل درخت رندر می‌شود. این درخت شامل تمام اطلاعات لازم برای مرحله بعدی است: تعیین دقیق محل نمایش هر یک از عناصر در صفحه. محاسبات هندسی برای بخش‌های مختلف DOM انجام می‌شود، از جمله محل شکست خطوط، که به موارد زبانی مانند چیدمان متن از چپ به راست یا راست به چپ (مانند عبری و عربی) بستگی دارد. شکستن خطوط نیازمند محاسبات دقیق است، زیرا مولفه‌های سبک‌دهی باید ویژگی‌های فونت را در نظر بگیرند. به‌وضوح مشخص است که فرایند تعیین چیدمان می‌تواند پیچیده شود